

Klase mokiniai

Sugeneruota Doxygen 1.13.2

1 Programos versijos	1
1.0.1 Versijos istorija:	1
1.1 Naudojimo instrukcija	1
1.1.1 Failų generavimas:	1
1.1.2 Failų paleidimas:	1
1.1.3 Sugeneruotų failų ištrynimasis:	2
1.1.4 Naudojimo pavyzdžiai:	2
1.2 Spartos analizė	2
1.2.1 Vector	2
1.2.2 Klasė "Vektorius"	2
1.2.3 List	2
1.2.4 Deque	2
1.2.5 Deque naudojant klasę (be flag)	2
1.2.6 Deque naudojant klasę (-O2)	3
1.2.7 Deque naudojant klasę (-O3)	3
1.2.8 Deque naudojant struct (-O3)	3
1.2.9 Vektorių pildymo spartos analizė	3
1.3 Sistemos parametrai	3
1.4 Išvados	3
2 Hierarchijos Indeksas	5
2.1 Klasių hierarchija	5
3 Klasės Indeksas	7
3.1 Klasės	7
4 Failo Indeksas	9
4.1 Failai	9
5 Klasės Dokumentacija	11
5.1 Stud Klasė	11
5.1.1 Smulkus aprašymas	13
5.1.2 Konstruktoriaus ir Destruktoriaus Dokumentacija	13
5.1.2.1 Stud() [1/4]	13
5.1.2.2 Stud() [2/4]	13
5.1.2.3 ~Stud()	14
5.1.2.4 Stud() [3/4]	14
5.1.2.5 Stud() [4/4]	14
5.1.3 Metodų Dokumentacija	15
5.1.3.1 getEgzaminas()	15
5.1.3.2 getMediana()	15
5.1.3.3 getPazymiai()	16
5.1.3.4 getVidurkis()	16
5.1.3.5 operator=() [1/2]	16

5.1.3.6 operator=()	[2/2]	16
5.1.3.7 setEgzaminas()		17
5.1.3.8 setMediana()		17
5.1.3.9 setPazymiai()		18
5.1.3.10 setVidurkis()		18
5.1.3.11 Skaityti()		18
5.1.4 Atributų Dokumentacija		19
5.1.4.1 egz		19
5.1.4.2 med		20
5.1.4.3 paz		20
5.1.4.4 vid		20
5.2 Vektorius< T > Klasė Šablonas		20
5.2.1 Tipo Aprašymo Dokumentacija		21
5.2.1.1 const_iterator		21
5.2.1.2 iterator		21
5.2.1.3 size_type		21
5.2.1.4 value_type		21
5.2.2 Konstruktoriaus ir Destruktoriaus Dokumentacija		21
5.2.2.1 Vektorius()	[1/5]	21
5.2.2.2 Vektorius()	[2/5]	22
5.2.2.3 Vektorius()	[3/5]	22
5.2.2.4 Vektorius()	[4/5]	22
5.2.2.5 Vektorius()	[5/5]	22
5.2.3 Metodų Dokumentacija		23
5.2.3.1 at()	[1/2]	23
5.2.3.2 at()	[2/2]	23
5.2.3.3 begin()	[1/2]	23
5.2.3.4 begin()	[2/2]	23
5.2.3.5 capacity()		24
5.2.3.6 clear()		24
5.2.3.7 empty()		25
5.2.3.8 end()	[1/2]	25
5.2.3.9 end()	[2/2]	26
5.2.3.10 erase()		26
5.2.3.11 operator=()	[1/2]	27
5.2.3.12 operator=()	[2/2]	27
5.2.3.13 operator[]()	[1/2]	27
5.2.3.14 operator[]()	[2/2]	28
5.2.3.15 pop_back()		28
5.2.3.16 push_back()		28
5.2.3.17 reallocate()		29
5.2.3.18 reserve()		30

5.2.3.19	resize()	31
5.2.3.20	size()	31
5.2.3.21	swap()	32
5.2.4	Atributų Dokumentacija	33
5.2.4.1	capacity_	33
5.2.4.2	data_	33
5.2.4.3	size_	33
5.3	Zmogus Klasė	33
5.3.1	Smulkus aprašymas	34
5.3.2	Konstruktoriaus ir Destruktoriaus Dokumentacija	34
5.3.2.1	Zmogus() [1/2]	34
5.3.2.2	Zmogus() [2/2]	35
5.3.2.3	~Zmogus()	35
5.3.3	Metodų Dokumentacija	35
5.3.3.1	getPavarde()	35
5.3.3.2	getVardas()	36
5.3.3.3	setPavarde()	36
5.3.3.4	setVardas()	36
5.3.3.5	Skaityti()	37
5.3.4	Atributų Dokumentacija	37
5.3.4.1	pav	37
5.3.4.2	vard	37
6	Failo Dokumentacija	39
6.1	README.md Failo Nuoroda	39
6.2	test/test.cpp Failo Nuoroda	39
6.2.1	Apibrėžimų Dokumentacija	40
6.2.1.1	CATCH_CONFIG_MAIN	40
6.2.2	Funkcijos Dokumentacija	40
6.2.2.1	TEST_CASE() [1/18]	40
6.2.2.2	TEST_CASE() [2/18]	40
6.2.2.3	TEST_CASE() [3/18]	41
6.2.2.4	TEST_CASE() [4/18]	41
6.2.2.5	TEST_CASE() [5/18]	41
6.2.2.6	TEST_CASE() [6/18]	42
6.2.2.7	TEST_CASE() [7/18]	42
6.2.2.8	TEST_CASE() [8/18]	42
6.2.2.9	TEST_CASE() [9/18]	43
6.2.2.10	TEST_CASE() [10/18]	43
6.2.2.11	TEST_CASE() [11/18]	43
6.2.2.12	TEST_CASE() [12/18]	44
6.2.2.13	TEST_CASE() [13/18]	44

6.2.2.14 TEST_CASE() [14/18]	44
6.2.2.15 TEST_CASE() [15/18]	45
6.2.2.16 TEST_CASE() [16/18]	45
6.2.2.17 TEST_CASE() [17/18]	45
6.2.2.18 TEST_CASE() [18/18]	45
6.3 vector/dll_header.h Failo Nuoroda	46
6.3.1 Apibrėžimų Dokumentacija	46
6.3.1.1 DLL_API	46
6.3.2 Funkcijos Dokumentacija	47
6.3.2.1 Faile()	47
6.4 dll_header.h	47
6.5 vector/functions.cpp Failo Nuoroda	48
6.5.1 Apibrėžimų Dokumentacija	48
6.5.1.1 EXPORTING_DLL	48
6.5.2 Funkcijos Dokumentacija	49
6.5.2.1 Auto()	49
6.5.2.2 Ekrane()	50
6.5.2.3 Faile()	50
6.5.2.4 GeneruotiFaila()	51
6.5.2.5 Manual()	51
6.5.2.6 Rusiuoti()	52
6.5.2.7 Semi()	53
6.5.2.8 Skirstymas1()	54
6.5.2.9 Skirstymas2()	55
6.5.2.10 Skirstymas3()	56
6.5.2.11 testRuleOfFive()	57
6.6 vector/header.h Failo Nuoroda	58
6.6.1 Funkcijos Dokumentacija	59
6.6.1.1 Auto()	59
6.6.1.2 Ekrane()	60
6.6.1.3 GeneruotiFaila()	61
6.6.1.4 Manual()	61
6.6.1.5 Rusiuoti()	62
6.6.1.6 Semi()	63
6.6.1.7 Skirstymas1()	64
6.6.1.8 Skirstymas2()	65
6.6.1.9 Skirstymas3()	66
6.6.1.10 testRuleOfFive()	67
6.7 header.h	68
6.8 vector/vector.h Failo Nuoroda	70
6.8.1 Funkcijos Dokumentacija	71
6.8.1.1 operator!=(())	71

6.8.1.2 operator<()	71
6.8.1.3 operator<=()	71
6.8.1.4 operator==()	71
6.8.1.5 operator>()	72
6.8.1.6 operator>=()	72
6.9 vector.h	72
6.10 vector/vektorius.cpp Failo Nuoroda	75
6.10.1 Funkcijos Dokumentacija	75
6.10.1.1 main()	75
Rodyklė	77

skyrius 1

Programos versijos

1.0.1 Versijos istorija:

1. **v.pradine** - Pradinė versija. Duomenų įvedimas ranka, surūšiuota išvestis ekrane.
 2. **v0.1** - Studentų ir pažymių generavimas, galimybė pasirinkti tarp vektoriaus ir masyvo naudojimo.
 3. **v0.2** - Failo skaitymas, išvestis į failą, rūšiavimas pagal naudotojo pasirinkimą.
 4. **v0.3** - Geresnė programavimo praktika, pridėtas klaidų apdorojimas (exception handling).
 5. **v0.4** - Studentų skirstymas į pogrupius, failų generavimas.
 6. **v1.0** - Pilna versija. Sukurtas Makefile, kelios versijos su skirtingais konteneriais, optimizacija.
 7. **v1.1** - Pertvarkyta struktūrą pakeičiant į klasę.
 8. **v1.2** - Panaudotas rule of five, implementuoti jo testai.
 9. **v1.5** - Klasė paskirstyta į abstrakčią ir derived klases.
 10. **v2.0** - dokumentacija, unit testai.
 11. **v3.0** - pilnai realizuota klasė (turi beveik pilną std::vector funkcionalumą)
-

1.1 Naudojimo instrukcija

Viskas vykdoma terminale.

1.1.1 Failų generavimas:

```
make all
```

1.1.2 Failų paleidimas:

Pereikite į vykdomųjų failų direktoriją:

```
cd bin
```

- **Vektoriai:** ./vector_program
- **List:** ./list_program
- **Deque:** ./deque_program

1.1.3 Sugeneruotų failų ištrynimasis:

Grįžkite į pradinę direktoriją:

```
cd ..
```

Valymo komanda:

```
make clean
```

1.1.4 Naudojimo pavyzdžiai:

1.2 Spartos analizė

1.2.1 Vector

Studentų skaičius	Failo skaitymas	Rūšiavimas	Paskirs- tymas	Spausdi- nimas	Iš viso	Paskirs- tymas (2)	Paskirs- tymas (3)
1k	0.015 s	0 s	0 s	0.011 s	0.026 s	0.028 s	0 s
10k	0.105 s	0.017 s	0.004 s	0.077 s	0.203 s	0.076 s	0.001 s
100k	0.392 s	0.138 s	0.021 s	0.771 s	1.322 s	180.122 s	0.012 s
1m	7.795 s	0.996 s	0.098 s	9.040 s	17.929 s	>5 min	0.048 s
10m	87.541 s	12.792 s	crash	88.551 s	188.884 s	crash	crash

1.2.2 Klasė "Vektorius"

Studentų skaičius	Failo skaitymas	Rūšiavimas	Paskirs- tymas	Spausdi- nimas	Iš viso	Paskirs- tymas (2)	Paskirs- tymas (3)
100k	0.230 s	0.027 s	0.032 s	0.288 s	0.577 s	2.474 s	0.031 s
1m	1.842 s	0.381 s	0.358 s	3.067 s	5.648 s	236.430 s	0.285 s
10m	24.155 s	5.036 s	2.785	28.227 s	60.204 s	>5 min	2.792 s

1.2.3 List

Studentų skaičius	Failo skaitymas	Rūšiavimas	Paskirs- tymas	Spausdi- nimas	Iš viso	Paskirs- tymas (2)	Paskirs- tymas (3)
1k	0.012 s	0 s	0 s	0.026 s	0.039 s	0 s	0 s
10k	0.117 s	0.013 s	0.006 s	0.107 s	0.244 s	0 s	0.001 s
100k	0.422 s	0.114 s	0.025 s	0.923 s	1.483 s	0.024 s	0.015 s
1m	7.131 s	0.579 s	0.216 s	9.120 s	17.046 s	0.182 s	0.397 s
10m	86.119 s	10.844 s	crash	89.815 s	188.504 s	1.726 s	4.288 s

1.2.4 Deque

Studentų skaičius	Failo skaitymas	Rūšiavimas	Paskirs- tymas	Spausdi- nimas	Iš viso	Paskirs- tymas (2)	Paskirs- tymas (3)
1k	0.015 s	0 s	0.002 s	0.029 s	0.046 s	0 s	0 s
10k	0.108 s	0.014 s	0.008 s	0.097 s	0.227 s	0 s	0.003 s
100k	0.407 s	0.155 s	0.029 s	0.919 s	1.510 s	0.025 s	0.012 s
1m	7.214 s	1.092 s	0.102 s	9.293 s	17.701 s	0.059 s	0.059 s
10m	85.353 s	12.767 s	crash	89.707 s	188.283 s	0.456 s	0.470 s

1.2.5 Deque naudojant klasę (be flag)

Studentų skaičius	Failo skaitymas	Rūšiavimas	Paskirstymas (3)	Spausdinimas	Iš viso
100k	1.231 s	0.367 s	0.031 s	0.944 s	2.574 s

1m	9.717 s	4.968 s	0.251 s	9.448 s	24.384 s
----	---------	---------	---------	---------	----------

.exe failo dydis - 372 KB

1.2.6 Deque naudojant klasę (-O2)

Studentų skaičius	Failo skaitymas	Rūšiavimas	Paskirstymas (3)	Spausdinimas	Iš viso
100k	0.954 s	0.100 s	0.007 s	0.892 s	1.954 s
1m	7.641 s	1.027 s	0.042 s	9.015 s	17.725 s

.exe failo dydis - 195 KB

1.2.7 Deque naudojant klasę (-O3)

Studentų skaičius	Failo skaitymas	Rūšiavimas	Paskirstymas (3)	Spausdinimas	Iš viso
100k	0.820 s	0.069 s	0.007 s	0.717 s	1.614 s
1m	6.125 s	0.875 s	0.051 s	7.403 s	14.455 s

.exe failo dydis - 190 KB

1.2.8 Deque naudojant struct (-O3)

Studentų skaičius	Failo skaitymas	Rūšiavimas	Paskirstymas (3)	Spausdinimas	Iš viso
100k	0.709 s	0.155 s	0.012 s	0.919 s	1.493 s
1m	7.214 s	1.092 s	0.059 s	8.895 s	17.260 s

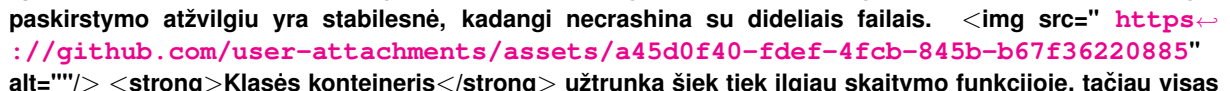
1.2.9 Vektorių pildymo spartos analizė

Vektoriaus dydis	10000	100000	1000000	10000000	100000000	Perskirstymai (100000000)
std::vector	73 micros	380 micros	2.670 ms	29.612 ms	209.857 ms	27
Klasė "Vektorius"	90 micros	282 micros	4.216 ms	39.728 ms	342.493 ms	27

1.3 Sistemos parametrai

- CPU: i7-13650HX
- RAM: 24GB 4800MHz
- Storage: NVMe M.2 SSD 1TB

1.4 Išvados

Vektoriai veikė prasčiausiai atminties atžvilgiu, crashino. **1 strategija** pasižymi prastu atminties išnaudojimu (su dideliais kiekiais taip pat sukėlė crash). **2 strategija** pasižymi didžiausiu spartumu (išskyrus su vektoriais). Naudojamos atminties kiekis tarp **deque ir list konteinerių** kito minimaliai, todėl spartos atžvilgiu **deque konteineris su strategija Nr. 2** yra geriausias pasirinkimas. Taip pat pastebime, kad **klasė** yra iki 20% spartesnė nei **struktūra**. Sukurta **klasė "Vektorius"** veikė lėčiau su studentų paskirstymais, tačiau sparčiau visur kitur nei **std::vector**. Tačiau **"Vektorius"** klasė paskirstymo atžvilgiu yra stabilesnė, kadangi necrashina su dideliais failais.  Klasės konteineris užtrunka šiek tiek ilgiau skaitymo funkcijoje, tačiau visas kitas funkcijas atlieka greičiau (skirtumas nėra didelis). Naudojant -O2 ir -O3 **Optimizavimo flag'us** matomas didelis spartos skirtumas palyginus be flag'o. Skirtumas tarp -O2 ir -O3 yra

minimalus, bet vistiek pastebime, kad -O3 veikia sparčiausiai. Su -O1 flag'u kilo problemų, yra errorų tarp naudojamos mingw32 versijos ir -O1 flag. <hr> @section autotoc_md27 Perdengtų metodų paaiškinimas Manual įvestis - sukuriamą laikina klasė, panaudojamas default konstruktorius. Vartotojas veda studentų vardus/pavardes arba "stop", kad užbaigtų įvedimą (nėra case sensitive). Vienu metu įvedami visi pažymiai atskirti tarpu, kurie yra saugojami vektoriuje. Įvedamas egzamino pažymys. Iš karto paskaičiuojami mediana ir vidurkiai, kad nereikėtų saugoti visų pažymių, kad sutaupyti atminties. Panaudojamas move konstruktorius perkelti iš laikinos klasės į naudojamą visoje programoje. Destruktorius sunaikina laikiną klasę. Semi įvestis - sukuriamą laikina klasė, panaudojamas default konstruktorius. Vartotojas veda studentų vardus/pavardes arba "stop", kad užbaigtų įvedimą (nėra case sensitive). Randomly sugeneruojami iki 10 pažymių. Iš karto paskaičiuojami mediana ir vidurkiai, kad nereikėtų saugoti visų pažymių, kad sutaupyti atminties. Panaudojamas move konstruktorius perkelti iš laikinos klasės į naudojamą visoje programoje. Destruktorius sunaikina laikiną klasę.

Auto įvestis - sukuriamą laikina klasė, panaudojamas default konstruktorius. Sugeneruojami iki 10 studentų (vardai, pavardės parenkami atsitiktinai iš vektorių), kiekvienas iki 10 pažymių. Iš karto paskaičiuojami mediana ir vidurkiai, kad nereikėtų saugoti visų pažymių, kad sutaupyti atminties. Panaudojamas move konstruktorius perkelti iš laikinos klasės į naudojamą visoje programoje. Destruktorius sunaikina laikiną klasę.

Ekrane išvestis - ekrane atspausdina visus surūšiuotus studentus (pagal vardą/pavardę/galutinį balą) ir jų galutinį balą vidurkį/medianą pagal vartotojo pasirinkimą. Rule of five nenaudojamas, kadangi nekuriama nauja klasė (nereikia default konstruktoriaus), neperkeliami ir nekopijuojami duomenys (nereikia move/copy konstruktoriaus/operatoriaus) ir nenaikinami laikini duomenys (nereikia destruktoriaus).

Faile išvestis - vartotojas pasirenka ar paskirstyti studentus į 2 atskirus failus pagal vidurkį. Skirstant į grupes sukuriami 2 nauji vektoriai, į kuriuos perkeliama duomenys naudojant move konstruktorių, priklausant nuo vartotojo pasirinktos rūšiavimo strategijos. Duomenys atspausdinami faile/failuose pagal vartotojo pasirinktą rūšiavimą. Panaudojamas destruktoriaus, kadangi vektoriams priklausė klasės nariai.

Kilus klausimams ar pastaboms, susisiekiel el. paštu: **nojus .petrusis@mif.stud.vu.lt**

skyrius 2

Hierarchijos Indeksas

2.1 Klasių hierarchija

Šis paveldėjimo sąrašas yra beveik surikiuotas abėcėlės tvarka:

Vektorius< T >	20
Zmogus	33
Stud	11

skyrius 3

Klasės Indeksas

3.1 Klasės

Klasės, struktūros, sąjungos ir sąsajos su trumpais aprašymais:

Stud	Išvestinė klasė	11
Vektorius< T >		20
Zmogus	Bazinė/abstrakti klasė	33

skyrius 4

Failo Indeksas

4.1 Failai

Visų failų sąrašas su trumpais aprašymais:

test/ test.cpp	39
vector/ dll_header.h	46
vector/ functions.cpp	48
vector/ header.h	58
vector/ vector.h	70
vector/ vektorius.cpp	75

skyrius 5

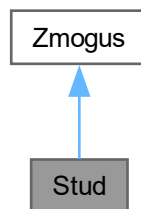
Klasės Dokumentacija

5.1 Stud Klasė

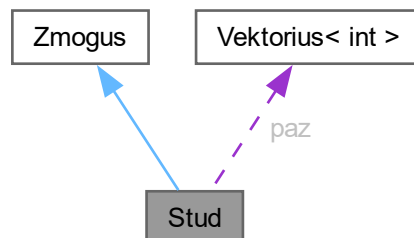
Išvestinė klasė.

```
#include <header.h>
```

Paveldimumo diagrama Stud:



Bendradarbiavimo diagrama Stud:



Vieši Metodai

- `Stud ()`

Numatytasis konstruktorius.

- **Stud** (const string &vardas, const string &pavarde, const **Vektorius**< int > &pazymiai, int egzaminas)
Konstruktorius su parametrais.
- **~Stud** ()
Destruktorius.
- **Stud** (const **Stud** &other)
Copy konstruktorius.
- **Stud** & **operator=** (const **Stud** &other)
Copy priskyrimo operatorius.
- **Stud** (**Stud** &&other) noexcept
Move konstruktorius.
- **Stud** & **operator=** (**Stud** &&other) noexcept
Move priskyrimo operatorius.
- **Vektorius**< int > **getPazymiai** () const
Getteriai.
- int **getEgzaminas** () const
- double **getVidurkis** () const
- double **getMediana** () const
- void **setPazymiai** (const **Vektorius**< int > &pazymiai)
Setteriai.
- void **setEgzaminas** (int egzaminas)
- void **setVidurkis** (double vidurkis)
- void **setMediana** (double mediana)
- void **Skaityti** (**Vektorius**< **Zmogus** * > &grupe, double &TotalTime) override
Perrašoma grynoji virtuali funkcija.

Vieši Metodai inherited from **Zmogus**

- **Zmogus** ()
Default konstruktorius.
- **Zmogus** (const string &vardas, const string &pavarde)
Konstruktorius su parametrais.
- virtual **~Zmogus** ()=default
Virtualus destruktoriaus.
- string **getVardas** () const
Getteriai.
- string **getPavarde** () const
- void **setVardas** (const string &vardas)
Setteriai.
- void **setPavarde** (const string &pavarde)

Privatūs Atributai

- **Vektorius**< int > **paz**
- int **egz**
- double **vid**
- double **med**

Additional Inherited Members

Apsaugoti Atributai inherited from **Zmogus**

- string **vard**
- string **pav**

5.1.1 Smulkus aprašymas

Išvestinė klasė.

5.1.2 Konstruktoriaus ir Destruktoriaus Dokumentacija

5.1.2.1 Stud() [1/4]

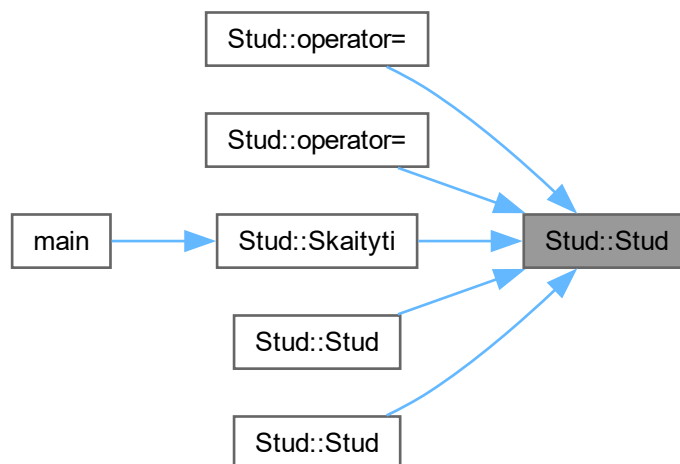
```
Stud::Stud () [inline]
```

Numatytasis konstruktorius.

Funkcijos kvietimo grafas:



Here is the caller graph for this function:



5.1.2.2 Stud() [2/4]

```
Stud::Stud (
    const string & vardas,
    const string & pavarde,
    const Vektorius< int > & pazymiai,
    int egzaminas) [inline]
```

Konstruktorius su parametrais.

Parametrai

<i>vardas</i>	Vardas.
---------------	---------

Parametrai

<i>pavarde</i>	Pavardė.
<i>pazymiai</i>	Pažymiai.
<i>egzaminas</i>	Egzamino rezultatas.

Funkcijos kvietimo grafas:



5.1.2.3 ~Stud()

`Stud::~~Stud ()`

Destruktorius.

5.1.2.4 Stud() [3/4]

`Stud::Stud (`

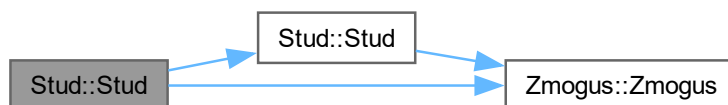
`const Stud & other) [inline]`

Copy konstruktorius.

Parametrai

<i>other</i>	Kitas objektas.
--------------	-----------------

Funkcijos kvietimo grafas:



5.1.2.5 Stud() [4/4]

`Stud::Stud (`

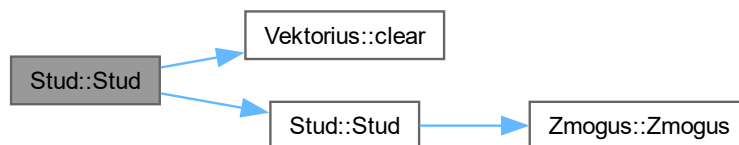
`Stud && other) [inline], [noexcept]`

Move konstruktorius.

Parametrai

<i>other</i>	Kitas objektas.
--------------	-----------------

Funkcijos kvietimo grafas:

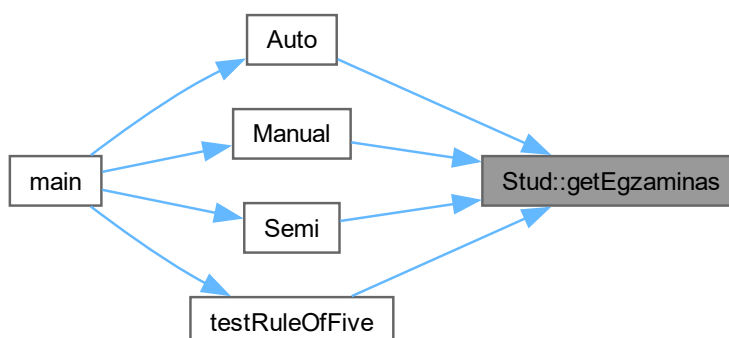


5.1.3 Metodų Dokumentacija

5.1.3.1 getEgzaminas()

```
int Stud::getEgzaminas () const [inline]
```

Here is the caller graph for this function:



5.1.3.2 getMediana()

```
double Stud::getMediana () const [inline]
```

Here is the caller graph for this function:



5.1.3.3 getPazymiai()

```
Vektorius< int > Stud::getPazymiai () const [inline]
Getteriai.
```

Here is the caller graph for this function:



5.1.3.4 getVidurkis()

```
double Stud::getVidurkis () const [inline]
```

5.1.3.5 operator=() [1/2]

```
Stud & Stud::operator= (
    const Stud & other) [inline]
```

Copy priskyrimo operatorius.

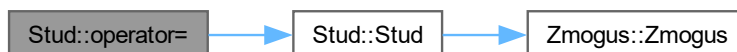
Parametrai

<i>other</i>	Kitas objektas.
--------------	-----------------

Gražina

Stud&

Funkcijos kvietimo grafas:



5.1.3.6 operator=() [2/2]

```
Stud & Stud::operator= (
    Stud && other) [inline], [noexcept]
```

Move priskyrimo operatorius.

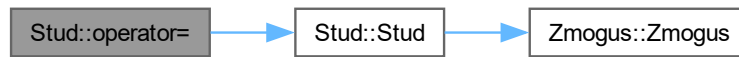
Parametrai

<i>other</i>	Kitas objektas.
--------------	-----------------

Gražina

Stud&

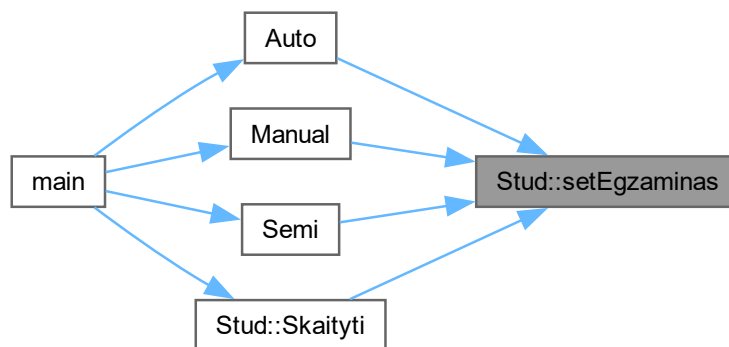
Funkcijos kvietimo grafas:



5.1.3.7 setEgzaminas()

```
void Stud::setEgzaminas (  
    int egzaminas) [inline]
```

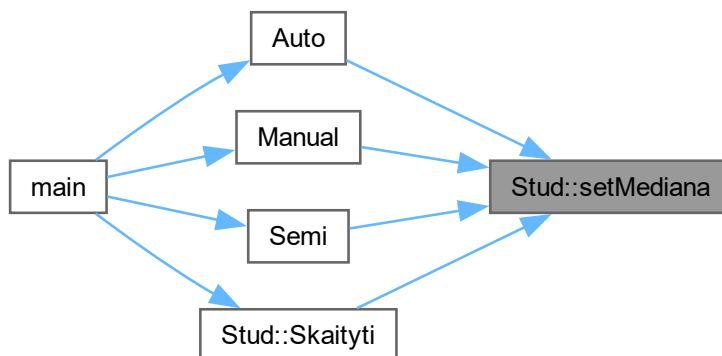
Here is the caller graph for this function:



5.1.3.8 setMediana()

```
void Stud::setMediana (  
    double mediana) [inline]
```

Here is the caller graph for this function:



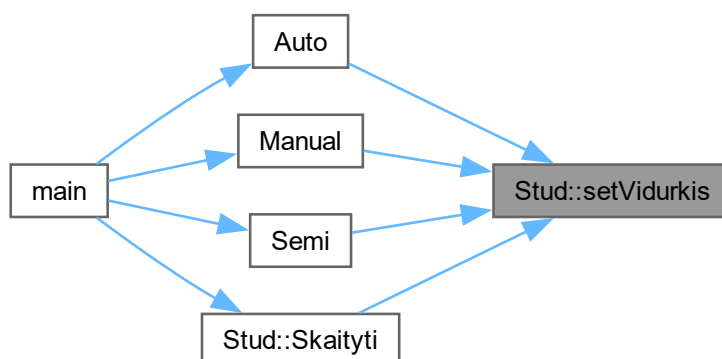
5.1.3.9 setPazymiai()

```
void Stud::setPazymiai (
    const Vektorius< int > & pazymiai) [inline]
Setteriai.
```

5.1.3.10 setVidurkis()

```
void Stud::setVidurkis (
    double vidurkis) [inline]
```

Here is the caller graph for this function:



5.1.3.11 Skaityti()

```
void Stud::Skaityti (
    Vektorius< Zmogus * > & grupe,
```

```
double & TotalTime) [override], [virtual]
```

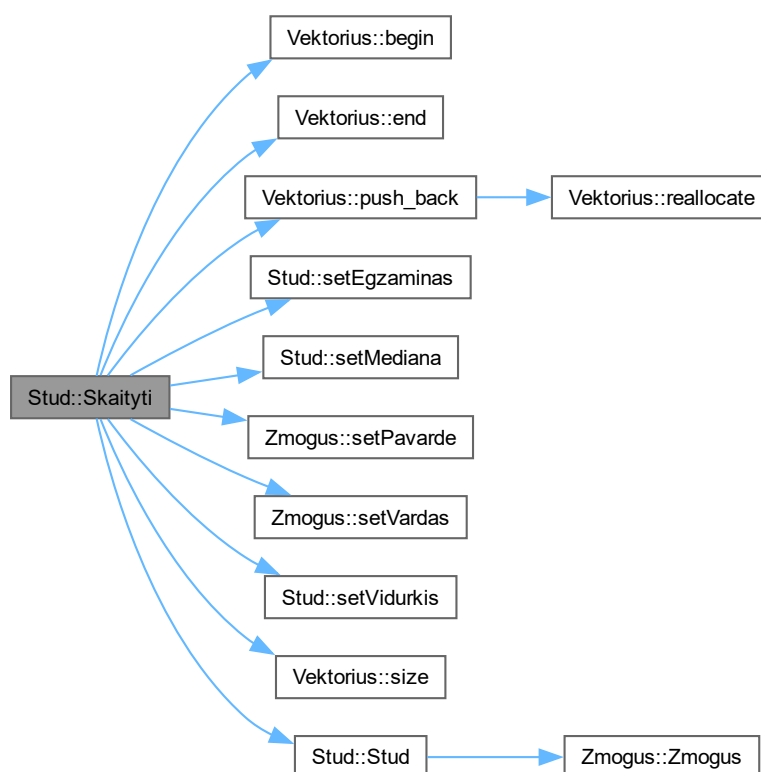
Perrašoma grynoji virtuali funkcija.

Parametrai

<i>grupe</i>	Studentų grupė.
<i>TotalTime</i>	Bendras operacijos laikas.

Realizuoja [Zmogus](#).

Funkcijos kvietimo grafas:



Here is the caller graph for this function:



5.1.4 Atributų Dokumentacija

5.1.4.1 egz

```
int Stud::egz [private]
```

5.1.4.2 med

```
double Stud::med [private]
```

5.1.4.3 paz

```
Vektorius<int> Stud::paz [private]
```

5.1.4.4 vid

```
double Stud::vid [private]
```

Dokumentacija šiai klasei sugeneruota iš šių failų:

- [vector/header.h](#)
- [vector/functions.cpp](#)

5.2 Vektorius< T > Klasė Šablonas

```
#include <vector.h>
```

Vieši Tipai

- using [value_type](#) = T
- using [size_type](#) = std::size_t
- using [iterator](#) = T *
- using [const_iterator](#) = const T *

Vieši Metodai

- [Vektorius](#) ()
- [Vektorius](#) ([size_type](#) count, const T &value=T())
- [Vektorius](#) (std::initializer_list< T > init)
- [Vektorius](#) (const [Vektorius](#) &other)
- [Vektorius](#) ([Vektorius](#) &&other) noexcept
- [Vektorius](#) & [operator=](#) (const [Vektorius](#) &other)
- [Vektorius](#) & [operator=](#) ([Vektorius](#) &&other) noexcept
- [size_type](#) [size](#) () const noexcept
- [size_type](#) [capacity](#) () const noexcept
- bool [empty](#) () const noexcept
- void [push_back](#) (const T &value)
- void [pop_back](#) ()
- void [clear](#) () noexcept
- void [swap](#) ([Vektorius](#) &other) noexcept
- void [resize](#) ([size_type](#) new_size, const T &value=T())
- void [reserve](#) ([size_type](#) new_capacity)
- [iterator](#) [erase](#) ([iterator](#) pos)
- T & [operator\[\]](#) ([size_type](#) index)
- const T & [operator\[\]](#) ([size_type](#) index) const
- T & [at](#) ([size_type](#) index)
- const T & [at](#) ([size_type](#) index) const
- [iterator](#) [begin](#) () noexcept
- [const_iterator](#) [begin](#) () const noexcept
- [iterator](#) [end](#) () noexcept
- [const_iterator](#) [end](#) () const noexcept

Privatūs Metodai

- void [reallocate](#) ([size_type](#) new_capacity)

Privatūs Atributai

- `std::unique_ptr< T[] >` `data_`
- `size_type` `size_`
- `size_type` `capacity_`

5.2.1 Tipo Aprašymo Dokumentacija**5.2.1.1 const_iterator**

```
template<typename T>
using Vektorius< T >::const_iterator = const T *
```

5.2.1.2 iterator

```
template<typename T>
using Vektorius< T >::iterator = T *
```

5.2.1.3 size_type

```
template<typename T>
using Vektorius< T >::size_type = std::size_t
```

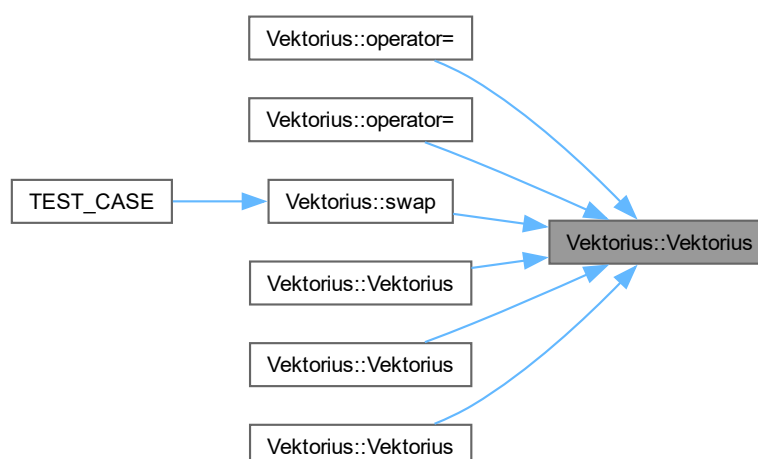
5.2.1.4 value_type

```
template<typename T>
using Vektorius< T >::value_type = T
```

5.2.2 Konstruktoriaus ir Destruktoriaus Dokumentacija**5.2.2.1 Vektorius() [1/5]**

```
template<typename T>
Vektorius< T >::Vektorius () [inline]
```

Here is the caller graph for this function:



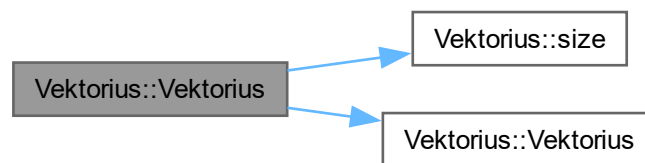
5.2.2.2 Vektorius() [2/5]

```
template<typename T>
Vektorius< T >::Vektorius (
    size_type count,
    const T & value = T()) [inline], [explicit]
```

5.2.2.3 Vektorius() [3/5]

```
template<typename T>
Vektorius< T >::Vektorius (
    std::initializer_list< T > init) [inline]
```

Funkcijos kvietimo grafas:



5.2.2.4 Vektorius() [4/5]

```
template<typename T>
Vektorius< T >::Vektorius (
    const Vektorius< T > & other) [inline]
```

Funkcijos kvietimo grafas:



5.2.2.5 Vektorius() [5/5]

```
template<typename T>
Vektorius< T >::Vektorius (
    Vektorius< T > && other) [inline], [noexcept]
```

Funkcijos kvietimo grafas:



5.2.3 Metodų Dokumentacija

5.2.3.1 at() [1/2]

```

template<typename T>
T & Vektorius< T >::at (
    size_type index) [inline]
  
```

Here is the caller graph for this function:



5.2.3.2 at() [2/2]

```

template<typename T>
const T & Vektorius< T >::at (
    size_type index) const [inline]
  
```

5.2.3.3 begin() [1/2]

```

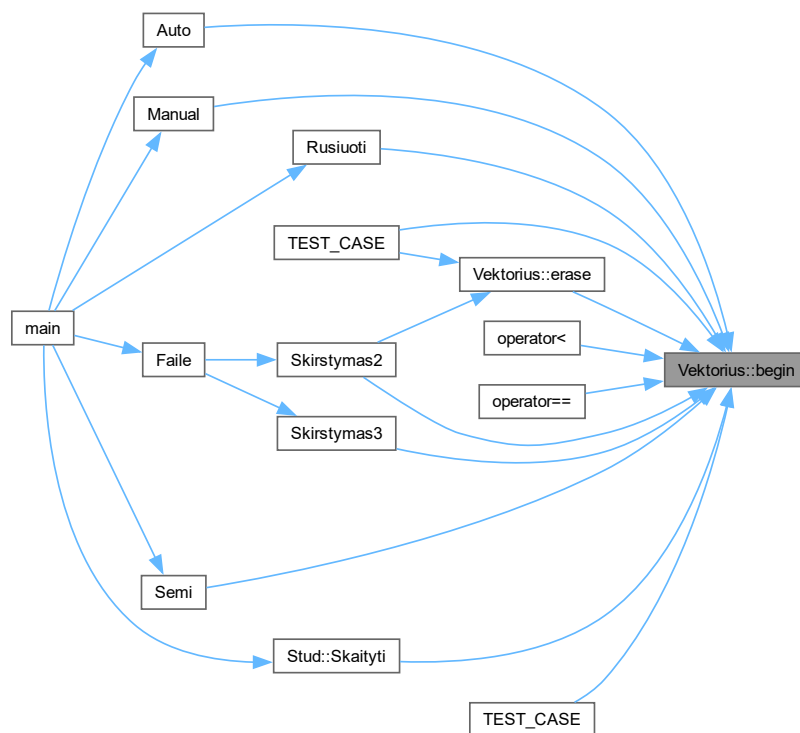
template<typename T>
const_iterator Vektorius< T >::begin () const [inline], [noexcept]
  
```

5.2.3.4 begin() [2/2]

```

template<typename T>
iterator Vektorius< T >::begin () [inline], [noexcept]
  
```

Here is the caller graph for this function:



5.2.3.5 capacity()

```
template<typename T>
size_type Vektorius< T >::capacity () const [inline], [noexcept]
```

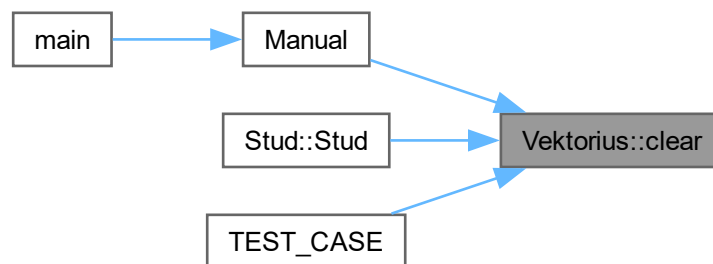
Here is the caller graph for this function:



5.2.3.6 clear()

```
template<typename T>
void Vektorius< T >::clear () [inline], [noexcept]
```


Here is the caller graph for this function:



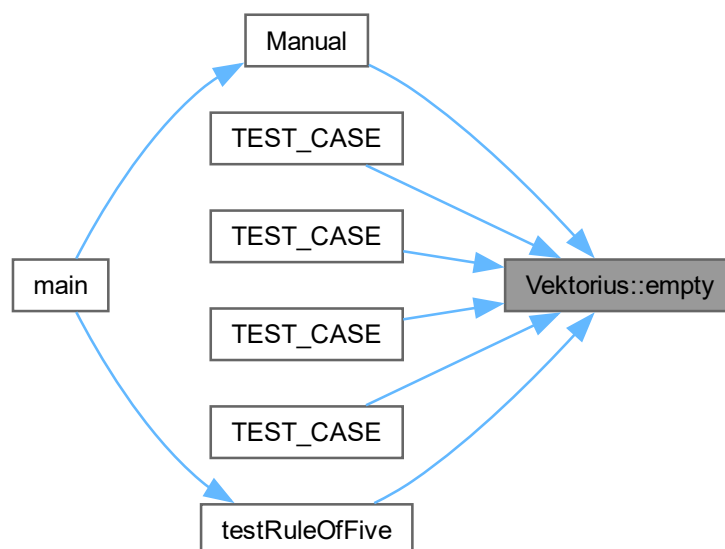
5.2.3.7 empty()

```

template<typename T>
bool Vektorius< T >::empty () const [inline], [noexcept]

```

Here is the caller graph for this function:



5.2.3.8 end() [1/2]

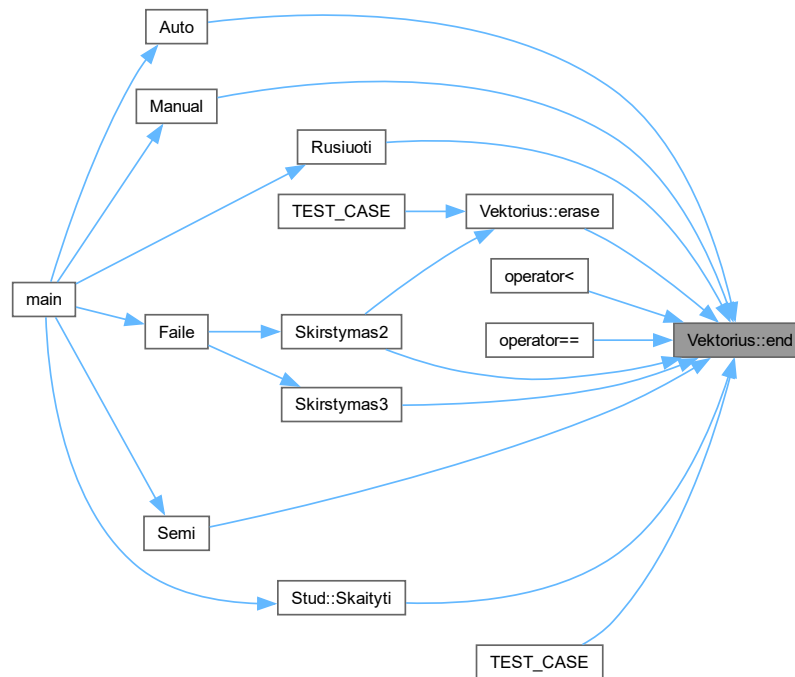
```

template<typename T>
const_iterator Vektorius< T >::end () const [inline], [noexcept]

```

5.2.3.9 end() [2/2]

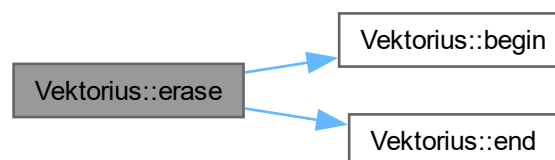
```
template<typename T>
iterator Vektorius< T >::end () [inline], [noexcept]
Here is the caller graph for this function:
```



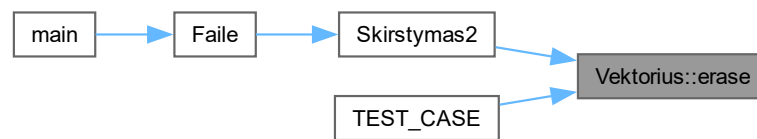
5.2.3.10 erase()

```
template<typename T>
iterator Vektorius< T >::erase (
    iterator pos) [inline]
```

Funkcijos kvietimo grafas:



Here is the caller graph for this function:



5.2.3.11 operator=() [1/2]

```

template<typename T>
Vektorius & Vektorius< T >::operator= (
    const Vektorius< T > & other) [inline]
  
```

Funkcijos kvietimo grafas:



5.2.3.12 operator=() [2/2]

```

template<typename T>
Vektorius & Vektorius< T >::operator= (
    Vektorius< T > && other) [inline], [noexcept]
  
```

Funkcijos kvietimo grafas:



5.2.3.13 operator[]() [1/2]

```

template<typename T>
T & Vektorius< T >::operator[] (
    size_type index) [inline]
  
```

5.2.3.14 operator[]() [2/2]

```
template<typename T>
const T & Vektorius< T >::operator[] (
    size_type index) const [inline]
```

5.2.3.15 pop_back()

```
template<typename T>
void Vektorius< T >::pop_back () [inline]
```

Here is the caller graph for this function:



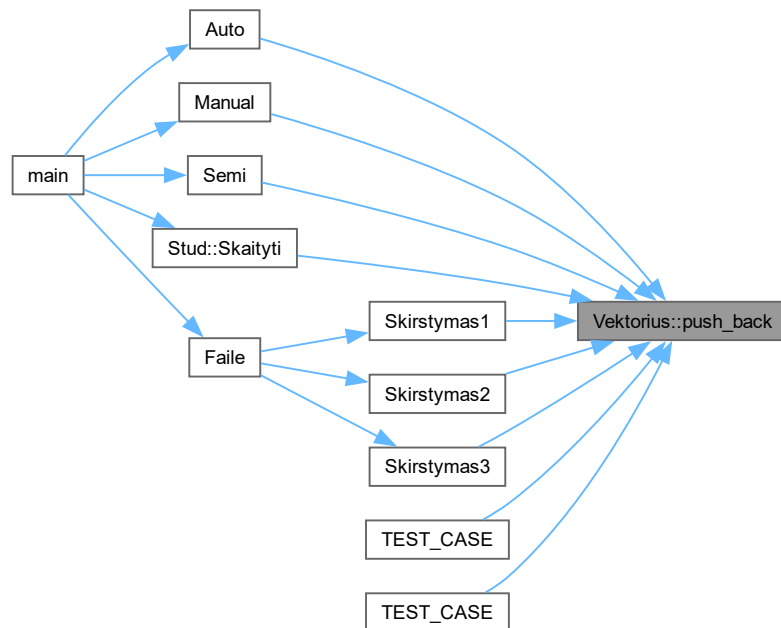
5.2.3.16 push_back()

```
template<typename T>
void Vektorius< T >::push_back (
    const T & value) [inline]
```

Funkcijos kvietimo grafas:



Here is the caller graph for this function:



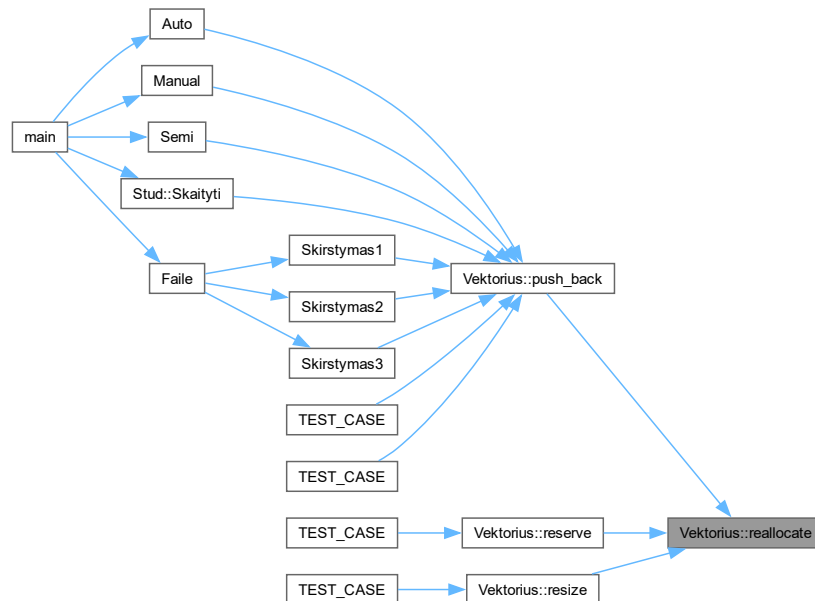
5.2.3.17 reallocate()

```

template<typename T>
void Vektorius< T >::reallocate (
    size_type new_capacity) [inline], [private]

```

Here is the caller graph for this function:



5.2.3.18 reserve()

```

template<typename T>
void Vektorius< T >::reserve (
    size_type new_capacity) [inline]
  
```

Funkcijos kvietimo grafas:



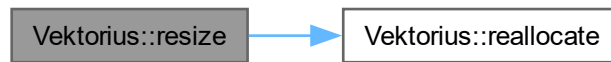
Here is the caller graph for this function:



5.2.3.19 resize()

```
template<typename T>
void Vektorius< T >::resize (
    size_type new_size,
    const T & value = T()) [inline]
```

Funkcijos kvietimo grafas:



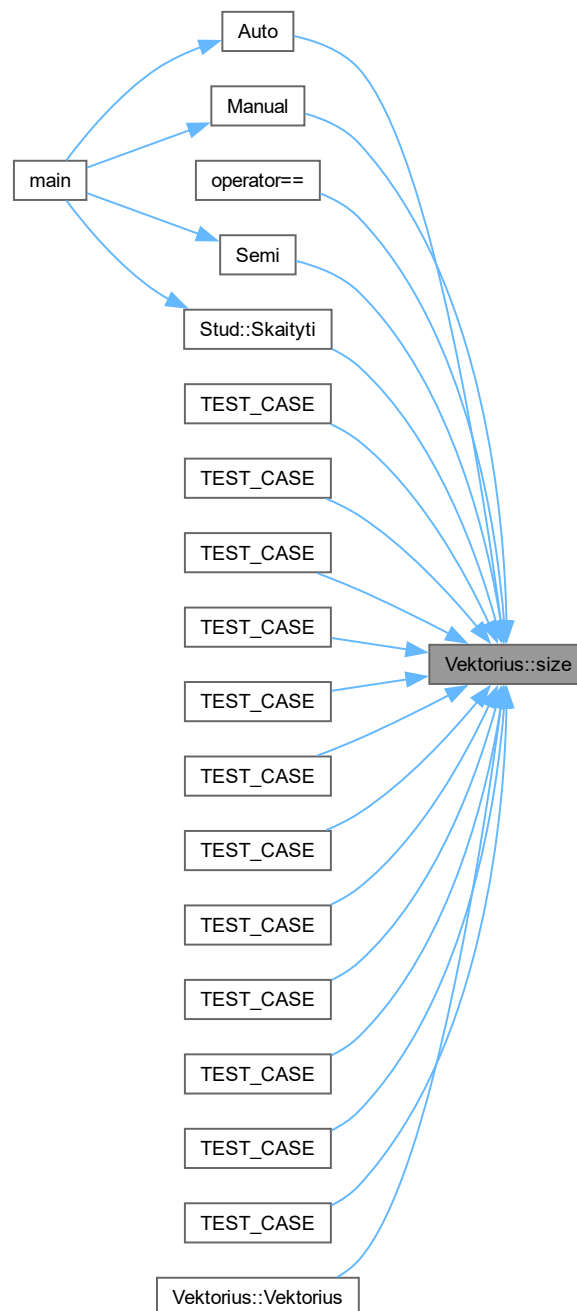
Here is the caller graph for this function:



5.2.3.20 size()

```
template<typename T>
size_type Vektorius< T >::size () const [inline], [noexcept]
```

Here is the caller graph for this function:

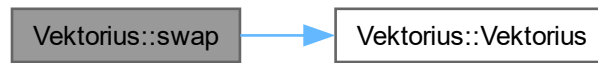


5.2.3.21 swap()

```

template<typename T>
void Vektorius< T >::swap (
    Vektorius< T > & other) [inline], [noexcept]
  
```


Funkcijos kvietimo grafas:



Here is the caller graph for this function:



5.2.4 Atributų Dokumentacija

5.2.4.1 capacity_

```
template<typename T>
size_type Vektorius< T >::capacity_ [private]
```

5.2.4.2 data_

```
template<typename T>
std::unique_ptr<T[]> Vektorius< T >::data_ [private]
```

5.2.4.3 size_

```
template<typename T>
size_type Vektorius< T >::size_ [private]
```

Dokumentacija šiai klasei sugeneruota iš šio failo:

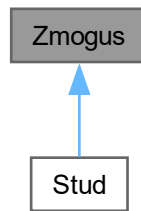
- vector/[vector.h](#)

5.3 Zmogus Klasė

Bazinė/abstrakti klasė.

```
#include <header.h>
```

Paveldimumo diagrama Zmogus:



Vieši Metodai

- `Zmogus ()`
Default konstruktorius.
- `Zmogus (const string &vardas, const string &pavarde)`
Konstruktorius su parametrais.
- `virtual ~Zmogus ()=default`
Virtualus destruktorius.
- `string getVardas () const`
Getteriai.
- `string getPavarde () const`
- `void setVardas (const string &vardas)`
Setteriai.
- `void setPavarde (const string &pavarde)`
- `virtual void Skaityti (Vektorius< Zmogus * > &grupe, double &TotalTime)=0`
Grynoji virtuali funkcija, kurią reikia įgyvendinti išvestinėse klasėse.

Apsaugoti Atributai

- `string vard`
- `string pav`

5.3.1 Smulkus aprašymas

Bazinė/abstrakti klasė.

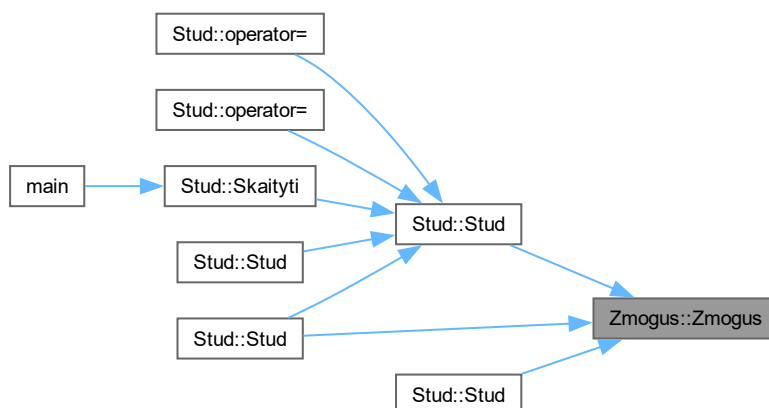
5.3.2 Konstruktoriaus ir Destruktoriaus Dokumentacija

5.3.2.1 Zmogus() [1/2]

```
Zmogus::Zmogus () [inline]
```

Default konstruktorius.

Here is the caller graph for this function:



5.3.2.2 Zmogus() [2/2]

```

Zmogus::Zmogus (
    const string & vardas,
    const string & pavarde) [inline]
  
```

Konstruktorius su parametrais.

Parametrai

<i>vardas</i>	Vardas.
<i>pavarde</i>	Pavardė.

5.3.2.3 ~Zmogus()

```
virtual Zmogus::~~Zmogus () [virtual], [default]
```

Virtualus destruktorius.

5.3.3 Metodų Dokumentacija

5.3.3.1 getPavarde()

```
string Zmogus::getPavarde () const [inline]
```

Here is the caller graph for this function:



5.3.3.2 getVardas()

```
string Zmogus::getVardas () const [inline]  
Getteriai.
```

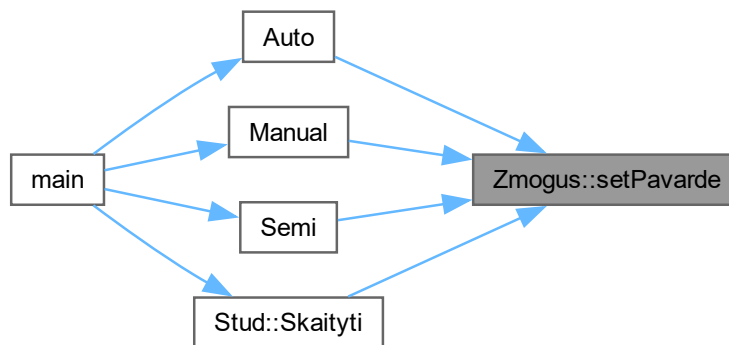
Here is the caller graph for this function:



5.3.3.3 setPavarde()

```
void Zmogus::setPavarde (  
    const string & pavarde) [inline]
```

Here is the caller graph for this function:

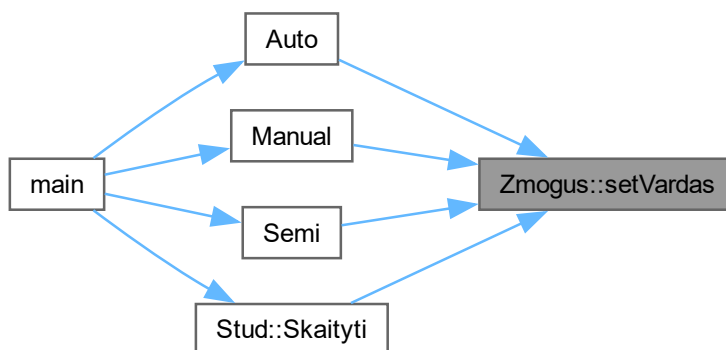


5.3.3.4 setVardas()

```
void Zmogus::setVardas (  
    const string & vardas) [inline]
```

Setteriai.

Here is the caller graph for this function:



5.3.3.5 Skaityti()

```
virtual void Zmogus::Skaityti (  
    Vektorius< Zmogus * > & grupe,  
    double & TotalTime) [pure virtual]
```

Grynoji virtuali funkcija, kurią reikia įgyvendinti išvestinėse klasėse.

Parametrai

<i>grupe</i>	Studentų grupė.
<i>TotalTime</i>	Bendras operacijos laikas.

Realizuota [Stud.](#)

5.3.4 Atributų Dokumentacija

5.3.4.1 pav

```
string Zmogus::pav [protected]
```

5.3.4.2 vard

```
string Zmogus::vard [protected]
```

Dokumentacija šiai klasei sugeneruota iš šio failo:

- [vector/header.h](#)

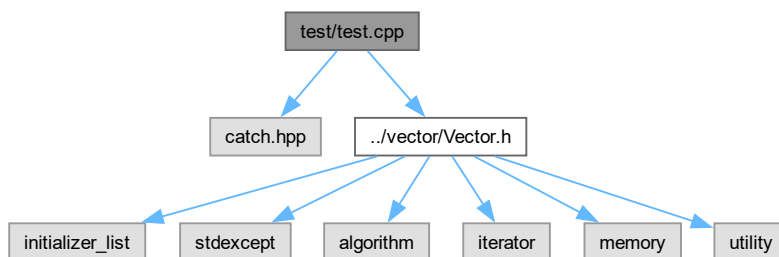
skyrius 6

Failo Dokumentacija

6.1 README.md Failo Nuoroda

6.2 test/test.cpp Failo Nuoroda

```
#include "catch.hpp"
#include "../vector/Vector.h"
}traukimo priklausomybių diagrama test.cpp:
```



Apibrėžimai

- `#define CATCH_CONFIG_MAIN`

Funkcijos

- `TEST_CASE` ("Vektorius - Default Constructor", "[vektorius]")
- `TEST_CASE` ("Vektorius - Constructor with Size and Default Value", "[vektorius]")
- `TEST_CASE` ("Vektorius - Initializer List Constructor", "[vektorius]")
- `TEST_CASE` ("Vektorius - Copy Constructor", "[vektorius]")
- `TEST_CASE` ("Vektorius - Move Constructor", "[vektorius]")
- `TEST_CASE` ("Vektorius - Copy Assignment", "[vektorius]")
- `TEST_CASE` ("Vektorius - Move Assignment", "[vektorius]")
- `TEST_CASE` ("Vektorius - Push Back", "[vektorius]")
- `TEST_CASE` ("Vektorius - Pop Back", "[vektorius]")
- `TEST_CASE` ("Vektorius - Clear", "[vektorius]")
- `TEST_CASE` ("Vektorius - Resize", "[vektorius]")
- `TEST_CASE` ("Vektorius - Reserve", "[vektorius]")
- `TEST_CASE` ("Vektorius - Erase", "[vektorius]")
- `TEST_CASE` ("Vektorius - Element Access", "[vektorius]")

- `TEST_CASE` ("Vektorius - Iterators", "[vektorius]")
- `TEST_CASE` ("Vektorius - Equality Operator", "[vektorius]")
- `TEST_CASE` ("Vektorius - Relational Operators", "[vektorius]")
- `TEST_CASE` ("Vektorius - Swap", "[vektorius]")

6.2.1 Apibrėžimų Dokumentacija

6.2.1.1 CATCH_CONFIG_MAIN

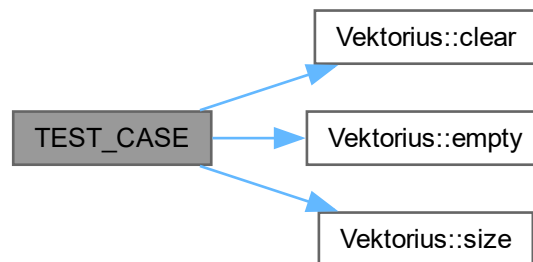
```
#define CATCH_CONFIG_MAIN
```

6.2.2 Funkcijos Dokumentacija

6.2.2.1 TEST_CASE() [1/18]

```
TEST_CASE (
    "Vektorius - Clear" ,
    "" [vektorius])
```

Funkcijos kvietimo grafas:



6.2.2.2 TEST_CASE() [2/18]

```
TEST_CASE (
    "Vektorius - Constructor with Size and Default Value" ,
    "" [vektorius])
```

Funkcijos kvietimo grafas:



6.2.2.3 TEST_CASE() [3/18]

```
TEST_CASE (
    "Vektorius - Copy Assignment" ,
    "" [vektorius])
```

Funkcijos kvietimo grafas:



6.2.2.4 TEST_CASE() [4/18]

```
TEST_CASE (
    "Vektorius - Copy Constructor" ,
    "" [vektorius])
```

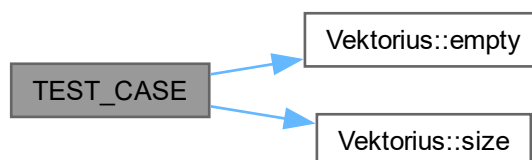
Funkcijos kvietimo grafas:



6.2.2.5 TEST_CASE() [5/18]

```
TEST_CASE (
    "Vektorius - Default Constructor" ,
    "" [vektorius])
```

Funkcijos kvietimo grafas:



6.2.2.6 TEST_CASE() [6/18]

```
TEST_CASE (
    "Vektorius - Element Access" ,
    "" [vektorius])
```

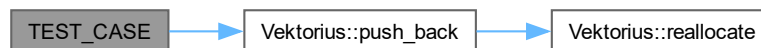
Funkcijos kvietimo grafas:



6.2.2.7 TEST_CASE() [7/18]

```
TEST_CASE (
    "Vektorius - Equality Operator" ,
    "" [vektorius])
```

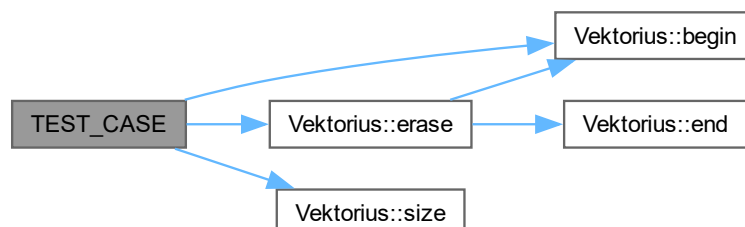
Funkcijos kvietimo grafas:



6.2.2.8 TEST_CASE() [8/18]

```
TEST_CASE (
    "Vektorius - Erase" ,
    "" [vektorius])
```

Funkcijos kvietimo grafas:



6.2.2.9 TEST_CASE() [9/18]

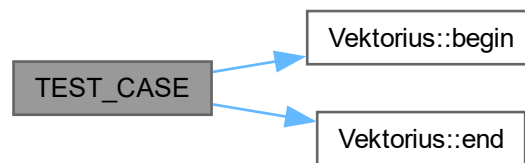
```
TEST_CASE (
    "Vektorius - Initializer List Constructor" ,
    "" [vektorius])
```

Funkcijos kvietimo grafas:

**6.2.2.10 TEST_CASE()** [10/18]

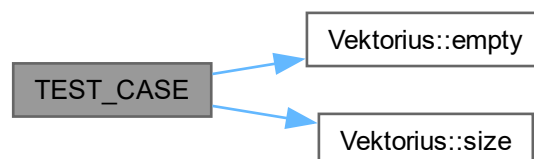
```
TEST_CASE (
    "Vektorius - Iterators" ,
    "" [vektorius])
```

Funkcijos kvietimo grafas:

**6.2.2.11 TEST_CASE()** [11/18]

```
TEST_CASE (
    "Vektorius - Move Assignment" ,
    "" [vektorius])
```

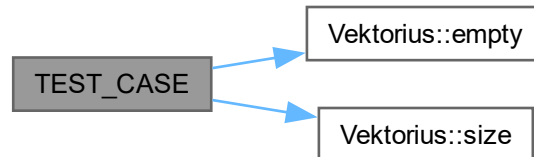
Funkcijos kvietimo grafas:



6.2.2.12 TEST_CASE() [12/18]

```
TEST_CASE (
    "Vektorius - Move Constructor" ,
    "" [vektorius])
```

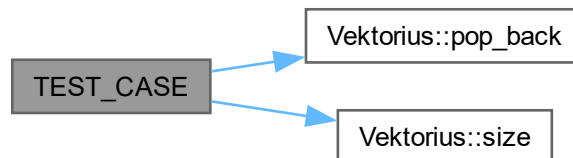
Funkcijos kvietimo grafas:



6.2.2.13 TEST_CASE() [13/18]

```
TEST_CASE (
    "Vektorius - Pop Back" ,
    "" [vektorius])
```

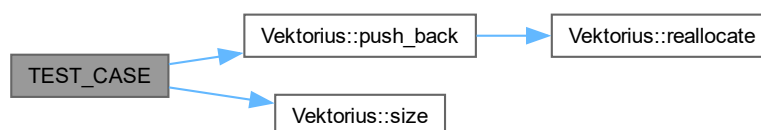
Funkcijos kvietimo grafas:



6.2.2.14 TEST_CASE() [14/18]

```
TEST_CASE (
    "Vektorius - Push Back" ,
    "" [vektorius])
```

Funkcijos kvietimo grafas:



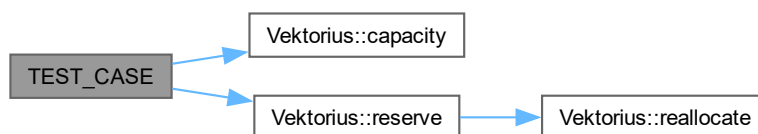
6.2.2.15 TEST_CASE() [15/18]

```
TEST_CASE (
    "Vektorius - Relational Operators" ,
    "" [vektorius])
```

6.2.2.16 TEST_CASE() [16/18]

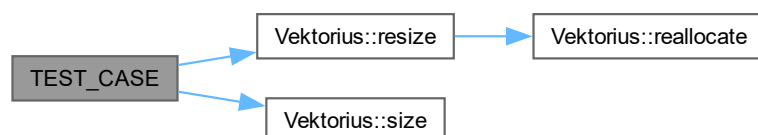
```
TEST_CASE (
    "Vektorius - Reserve" ,
    "" [vektorius])
```

Funkcijos kvietimo grafas:

**6.2.2.17 TEST_CASE() [17/18]**

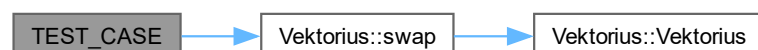
```
TEST_CASE (
    "Vektorius - Resize" ,
    "" [vektorius])
```

Funkcijos kvietimo grafas:

**6.2.2.18 TEST_CASE() [18/18]**

```
TEST_CASE (
    "Vektorius - Swap" ,
    "" [vektorius])
```

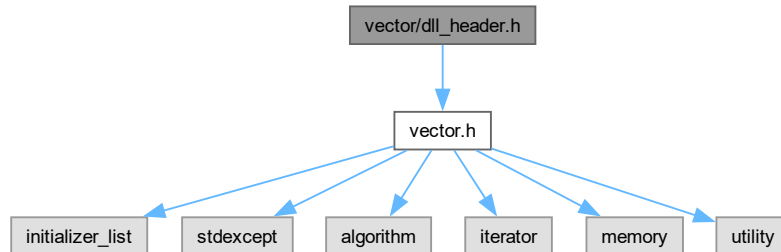
Funkcijos kvietimo grafas:



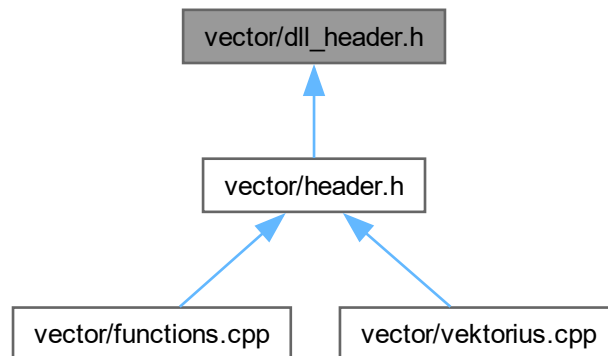
6.3 vector/dll_header.h Failo Nuoroda

```
#include "vector.h"
```

Įtraukimo priklausomybių diagrama dll_header.h:



Šis grafas rodo, kuris failas tiesiogiai ar netiesiogiai įtraukia šį failą:



Apibrėžimai

- `#define DLL_API __declspec(dllimport)`

Funkcijos

- `DLL_API void Faile (Vektorius< Zmogus * > &grupe, char gal, double &TotalTime)`
Exported function for the DLL.

6.3.1 Apibrėžimų Dokumentacija

6.3.1.1 DLL_API

```
#define DLL_API __declspec(dllimport)
```

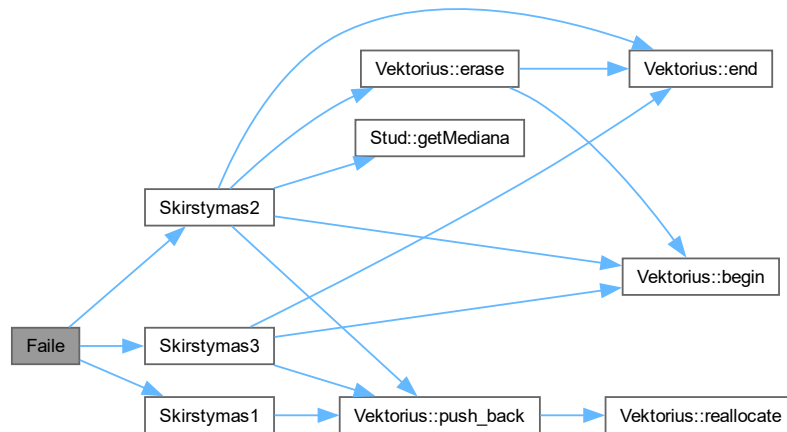
6.3.2 Funkcijos Dokumentacija

6.3.2.1 Faile()

```
DLL_API void Faile (
    Vektorius< Zmogus * > &grupe,
    char gal,
    double &TotalTime)
```

Exported function for the DLL.

This function will be exported and can be used by other applications. Funkcijos kvietimo grafas:



Here is the caller graph for this function:



6.4 dll_header.h

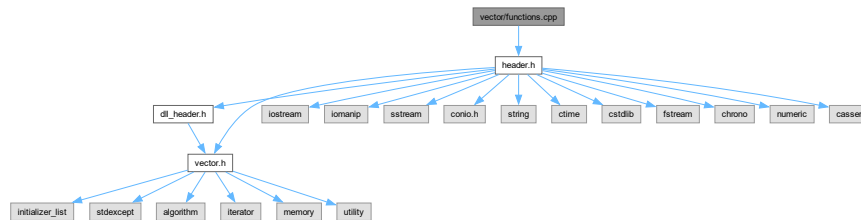
Eiti į šio failo dokumentaciją.

```
00001 #pragma once
00002
00003 #include "vector.h" // Ensure this is necessary
00004
00005 #ifdef EXPORTING_DLL
00006 #define DLL_API __declspec(dllexport)
00007 #else
00008 #define DLL_API __declspec(dllimport)
00009 #endif
00010
00011 // Forward declaration of Zmogus
00012 class Zmogus;
00013
00019 DLL_API void Faile(Vektorius<Zmogus *> &grupe, char gal, double &TotalTime);
```

6.5 vector/functions.cpp Failo Nuoroda

```
#include "header.h"
```

Itraukimo priklausomybių diagrama functions.cpp:



Apibrėžimai

- #define `EXPORTING_DLL`

Funkcijos

- void `Manual` (`Stud` &laik, `Vektorius`< `Zmogus` * > &grupe)
Funkcija studentų duomenų įvedimui rankiniu būdu.
- void `Semi` (`Stud` &laik, `Vektorius`< `Zmogus` * > &grupe)
Funkcija studentų duomenų įvedimui pusiau automatinio būdu.
- void `Auto` (`Vektorius`< `Zmogus` * > &grupe)
Funkcija studentų duomenų generavimui automatiškai.
- void `Ekrane` (`Vektorius`< `Zmogus` * > &grupe, char gal, double &TotalTime)
Funkcija studentų duomenų išvedimui į ekraną.
- void `Skirstymas1` (`Vektorius`< `Zmogus` * > &grupe, char gal, `Vektorius`< `Stud` > &vargsiukai, `Vektorius`< `Stud` > &galvociiai, double &TotalTime)
Funkcija studentų skirstymui į dvi grupes pagal vidurkį arba medianą.
- void `Skirstymas2` (`Vektorius`< `Zmogus` * > &grupe, char gal, `Vektorius`< `Stud` > &vargsiukai, double &TotalTime)
Funkcija studentų skirstymui į dvi grupes su pašalinimu iš pradinės grupės.
- void `Skirstymas3` (`Vektorius`< `Zmogus` * > &grupe, char gal, `Vektorius`< `Stud` > &vargsiukai, `Vektorius`< `Stud` > &galvociiai, double &TotalTime)
Funkcija studentų skirstymui į dvi grupes naudojant `std::partition`.
- `DLL_API` void `Faile` (`Vektorius`< `Zmogus` * > &grupe, char gal, double &TotalTime)
Exported function for the DLL.
- void `Rusiuoti` (`Vektorius`< `Zmogus` * > &grupe, char rusiavimas, char gal, double &TotalTime)
Funkcija studentų rikiavimui pagal pasirinktą kriterijų.
- void `GeneruotiFaila` (double &TotalTime)
Funkcija studentų failo generavimui.
- void `testRuleOfFive` ()
Funkcija Rule of Five taisyklės testavimui.

6.5.1 Apibrėžimų Dokumentacija

6.5.1.1 EXPORTING_DLL

```
#define EXPORTING_DLL
```


6.5.2 Funkcijos Dokumentacija

6.5.2.1 Auto()

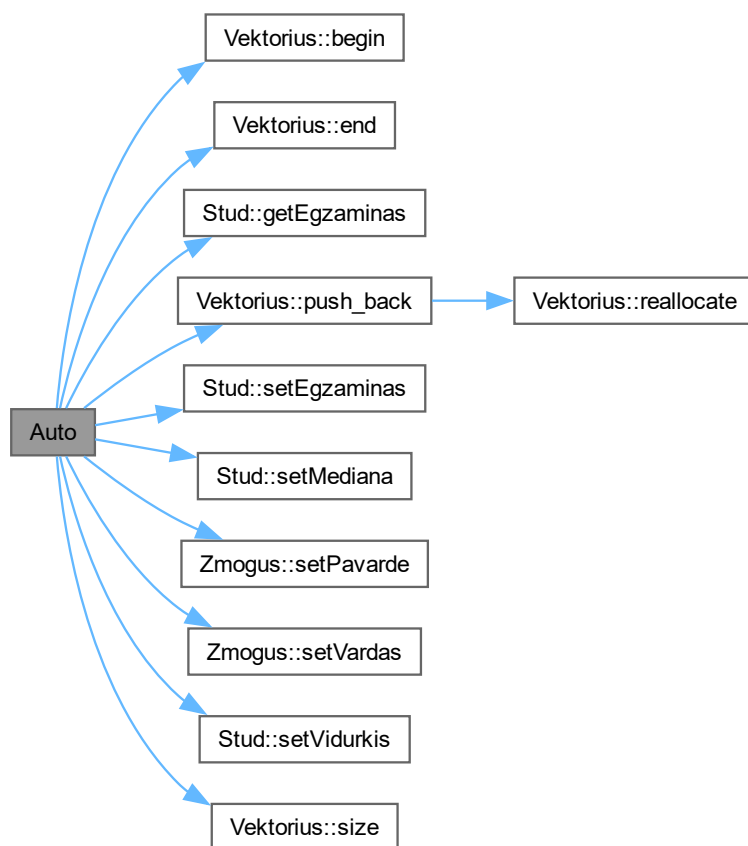
```
void Auto (  
    Vektorius< Zmogus * > & grupe)
```

Funkcija studentų duomenų generavimui automatiškai.

Parametrai

<i>grupe</i>	Studentų grupė.
--------------	-----------------

Funkcijos kvietimo grafas:



Here is the caller graph for this function:



6.5.2.2 Ekrane()

```
void Ekrane (
    Vektorius< Zmogus * > & grupe,
    char gal,
    double & TotalTime)
```

Funkcija studentų duomenų išvedimui į ekraną.

Parametrai

<i>grupe</i>	Studentų grupė.
<i>gal</i>	Pasirinkimas, ar naudoti vidurkį ('0') ar medianą ('1').
<i>TotalTime</i>	Bendras operacijos laikas.

Here is the caller graph for this function:

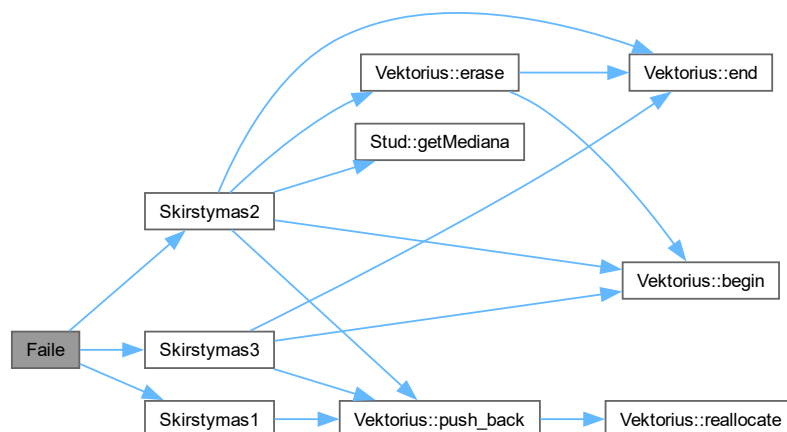


6.5.2.3 Faile()

```
DLL_API void Faile (
    Vektorius< Zmogus * > & grupe,
    char gal,
    double & TotalTime)
```

Exported function for the DLL.

This function will be exported and can be used by other applications. Funkcijos kvietimo grafas:



Here is the caller graph for this function:



6.5.2.4 GeneruotiFaila()

```
void GeneruotiFaila (  
    double & TotalTime)
```

Funkcija studentų failo generavimui.

Parametrai

<i>TotalTime</i>	Bendras operacijos laikas.
------------------	----------------------------

Here is the caller graph for this function:



6.5.2.5 Manual()

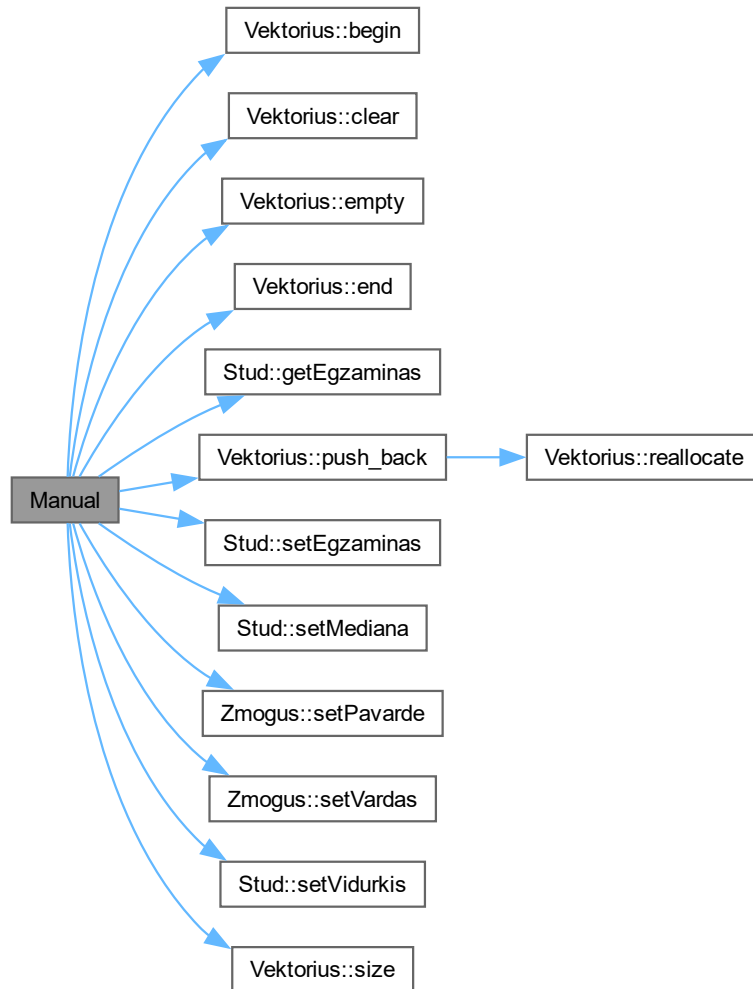
```
void Manual (  
    Stud & laik,  
    Vektorius< Zmogus * > & grupe)
```

Funkcija studentų duomenų įvedimui rankiniu būdu.

Parametrai

<i>laik</i>	Laikinas studento objektas.
<i>grupe</i>	Studentų grupė.

Funkcijos kvietimo grafas:



Here is the caller graph for this function:



6.5.2.6 Rusiuoti()

```
void Rusiuoti (
```

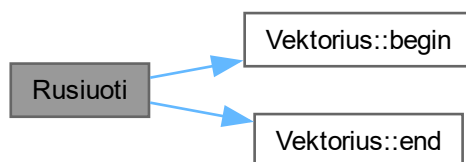
```
Vektorius< Zmogus * > & grupe,
char rusiavimas,
char gal,
double & TotalTime)
```

Funkcija studentų rikiavimui pagal pasirinktą kriterijų.

Parametrai

<i>grupe</i>	Studentų grupė.
<i>rusiavimas</i>	Rikiavimo kriterijus ('v' - vardas, 'p' - pavardė, 'g' - galutinis rezultatas).
<i>gal</i>	Pasirinkimas, ar naudoti vidurkį ('0') ar medianą ('1').
<i>TotalTime</i>	Bendras operacijos laikas.

Funkcijos kvietimo grafas:



Here is the caller graph for this function:



6.5.2.7 Semi()

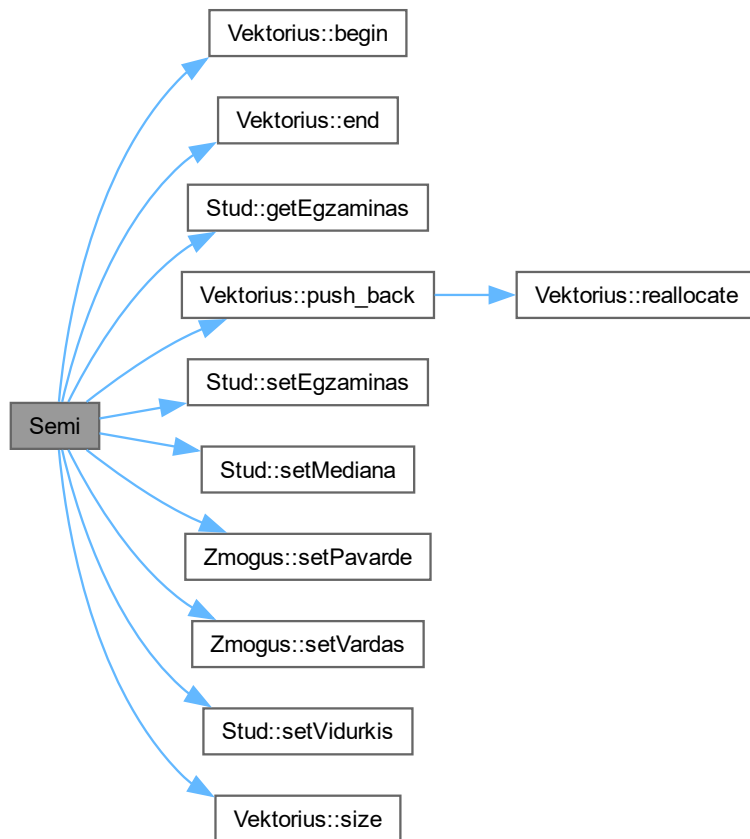
```
void Semi (
    Stud & laik,
    Vektorius< Zmogus * > & grupe)
```

Funkcija studentų duomenų įvedimui pusiau automatinio būdu.

Parametrai

<i>laik</i>	Laikinas studento objektas.
<i>grupe</i>	Studentų grupė.

Funkcijos kvietimo grafas:



Here is the caller graph for this function:



6.5.2.8 Skirstymas1()

```

void Skirstymas1 (
    Vektorius< Zmogus * > & grupe,
    char gal,
    Vektorius< Stud > & vargsiukai,

```

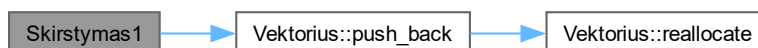
```
Vektorius< Stud > & galvociai,
double & TotalTime)
```

Funkcija studentų skirstymui į dvi grupes pagal vidurkį arba medianą.

Parametrai

<i>grupe</i>	Studentų grupė.
<i>gal</i>	Pasirinkimas, ar naudoti vidurkį ('0') ar medianą ('1').
<i>vargsiukai</i>	Grupė studentų, kurių rezultatai mažesni nei 5.
<i>galvociai</i>	Grupė studentų, kurių rezultatai didesni arba lygūs 5.
<i>TotalTime</i>	Bendras operacijos laikas.

Funkcijos kvietimo grafas:



Here is the caller graph for this function:



6.5.2.9 Skirstymas2()

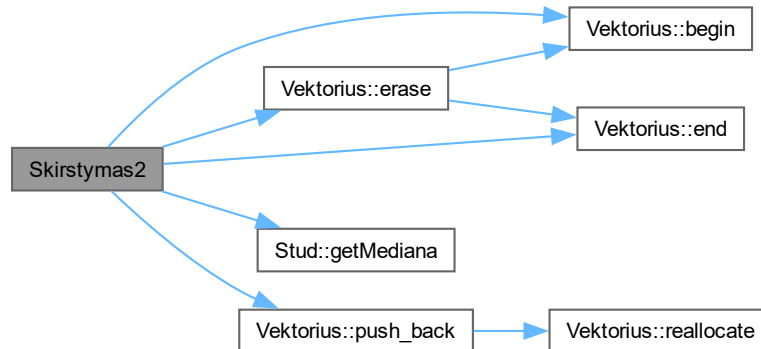
```
void Skirstymas2 (
    Vektorius< Zmogus * > & grupe,
    char gal,
    Vektorius< Stud > & vargsiukai,
    double & TotalTime)
```

Funkcija studentų skirstymui į dvi grupes su pašalinimu iš pradinės grupės.

Parametrai

<i>grupe</i>	Studentų grupė.
<i>gal</i>	Pasirinkimas, ar naudoti vidurkį ('0') ar medianą ('1').
<i>vargsiukai</i>	Grupė studentų, kurių rezultatai mažesni nei 5.
<i>TotalTime</i>	Bendras operacijos laikas.

Funkcijos kvietimo grafas:



Here is the caller graph for this function:



6.5.2.10 Skirstymas3()

```

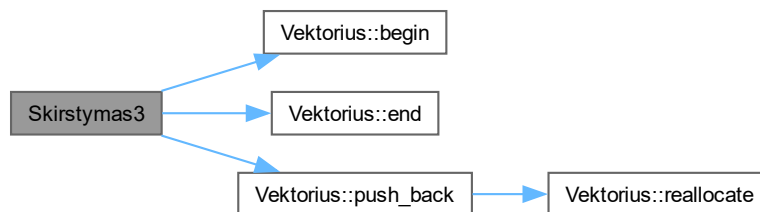
void Skirstymas3 (
    Vektorius< Zmogus * > & grupe,
    char gal,
    Vektorius< Stud > & vargsiukai,
    Vektorius< Stud > & galvociai,
    double & TotalTime)
  
```

Funkcija studentų skirstymui į dvi grupes naudojant `std::partition`.

Parametrai

<i>grupe</i>	Studentų grupė.
<i>gal</i>	Pasirinkimas, ar naudoti vidurkį ('0') ar medianą ('1').
<i>vargsiukai</i>	Grupė studentų, kurių rezultatai mažesni nei 5.
<i>galvociai</i>	Grupė studentų, kurių rezultatai didesni arba lygūs 5.
<i>TotalTime</i>	Bendras operacijos laikas.

Funkcijos kvietimo grafas:



Here is the caller graph for this function:

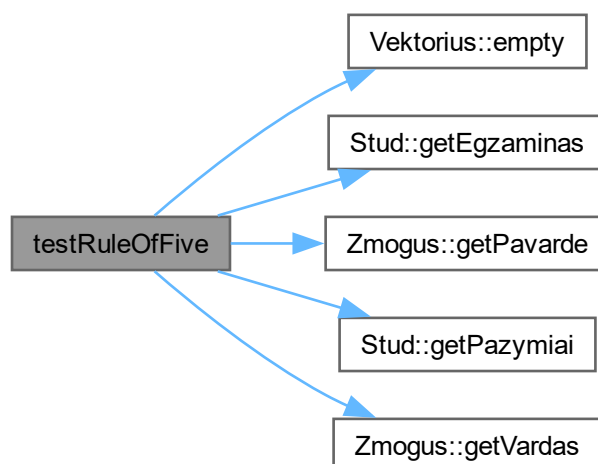


6.5.2.11 testRuleOfFive()

```
void testRuleOfFive ()
```

Funkcija Rule of Five taisyklės testavimui.

Funkcijos kvietimo grafas:



Here is the caller graph for this function:



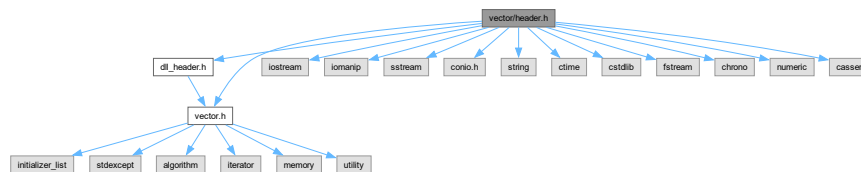
6.6 vector/header.h Failo Nuoroda

```

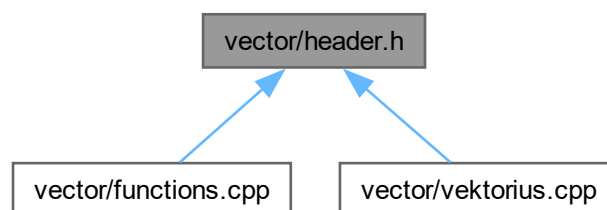
#include "dll_header.h"
#include "vector.h"
#include <iostream>
#include <iomanip>
#include <sstream>
#include <conio.h>
#include <string>
#include <ctime>
#include <cstdlib>
#include <fstream>
#include <chrono>
#include <numeric>
#include <cassert>

```

Įtraukimo priklausomybių diagrama header.h:



Šis grafas rodo, kuris failas tiesiogiai ar netiesiogiai įtraukia šį failą:



Klasės

- class [Zmogus](#)

Bazinė/abstrakti klasė.

- class [Stud](#)

Išvestinė klasė.

Funkcijos

- void [Manual](#) ([Stud](#) &laik, [Vektorius](#)< [Zmogus](#) * > &grupe)
Funkcija studentų duomenų įvedimui rankiniu būdu.
- void [Semi](#) ([Stud](#) &laik, [Vektorius](#)< [Zmogus](#) * > &grupe)
Funkcija studentų duomenų įvedimui pusiau automatinio būdu.
- void [Auto](#) ([Vektorius](#)< [Zmogus](#) * > &grupe)
Funkcija studentų duomenų generavimui automatiškai.
- void [Ekrane](#) ([Vektorius](#)< [Zmogus](#) * > &grupe, char gal, double &TotalTime)
Funkcija studentų duomenų išvedimui į ekraną.
- void [Skirstymas1](#) ([Vektorius](#)< [Zmogus](#) * > &grupe, char gal, [Vektorius](#)< [Stud](#) > &vargsiukai, [Vektorius](#)< [Stud](#) > &galvociiai, double &TotalTime)
Funkcija studentų skirstymui į dvi grupes pagal vidurkį arba medianą.
- void [Skirstymas2](#) ([Vektorius](#)< [Zmogus](#) * > &grupe, char gal, [Vektorius](#)< [Stud](#) > &vargsiukai, double &TotalTime)
Funkcija studentų skirstymui į dvi grupes su pašalinimu iš pradinės grupės.
- void [Skirstymas3](#) ([Vektorius](#)< [Zmogus](#) * > &grupe, char gal, [Vektorius](#)< [Stud](#) > &vargsiukai, [Vektorius](#)< [Stud](#) > &galvociiai, double &TotalTime)
Funkcija studentų skirstymui į dvi grupes naudojant std::partition.
- void [Rusiuoti](#) ([Vektorius](#)< [Zmogus](#) * > &grupe, char rusiavimas, char gal, double &TotalTime)
Funkcija studentų rikiavimui pagal pasirinktą kriterijų.
- void [GeneruotiFaila](#) (double &TotalTime)
Funkcija studentų failo generavimui.
- void [testRuleOfFive](#) ()
Funkcija Rule of Five taisyklės testavimui.

6.6.1 Funkcijos Dokumentacija

6.6.1.1 Auto()

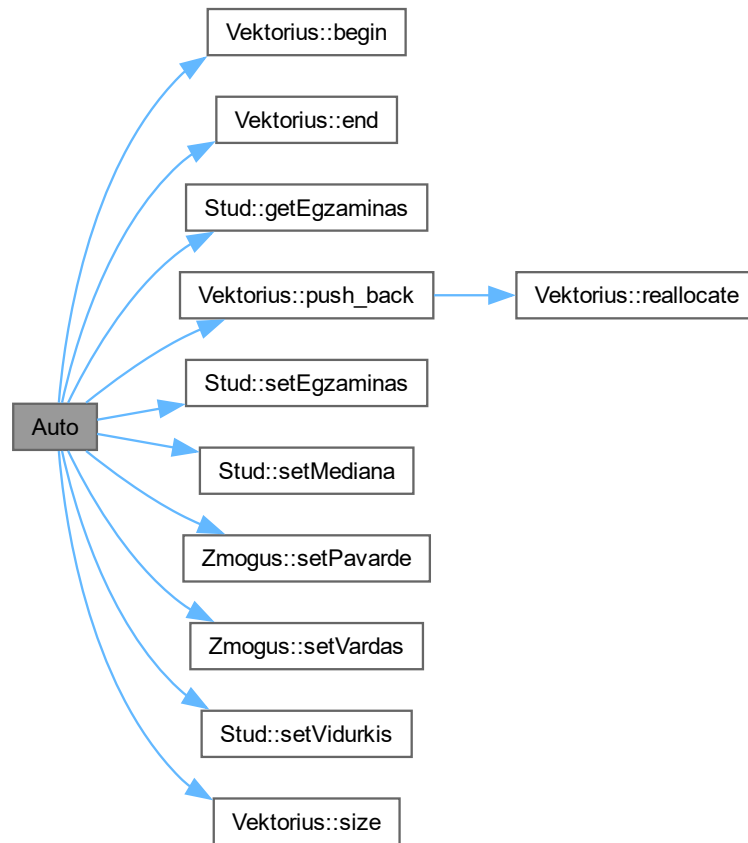
```
void Auto (
    Vektorius< Zmogus * > & grupe)
```

Funkcija studentų duomenų generavimui automatiškai.

Parametrai

<i>grupe</i>	Studentų grupė.
--------------	-----------------

Funkcijos kvietimo grafas:



Here is the caller graph for this function:



6.6.1.2 Ekrane()

```

void Ekrane (
    Vektorius< Zmogus * > & grupe,
    char gal,
    double & TotalTime)

```

Funkcija studentų duomenų išvedimui į ekraną.

Parametrai

<i>grupe</i>	Studentų grupė.
<i>gal</i>	Pasirinkimas, ar naudoti vidurkį ('0') ar medianą ('1').
<i>TotalTime</i>	Bendras operacijos laikas.

Here is the caller graph for this function:



6.6.1.3 GeneruotiFaila()

```
void GeneruotiFaila (  
    double & TotalTime)
```

Funkcija studentų failo generavimui.

Parametrai

<i>TotalTime</i>	Bendras operacijos laikas.
------------------	----------------------------

Here is the caller graph for this function:



6.6.1.4 Manual()

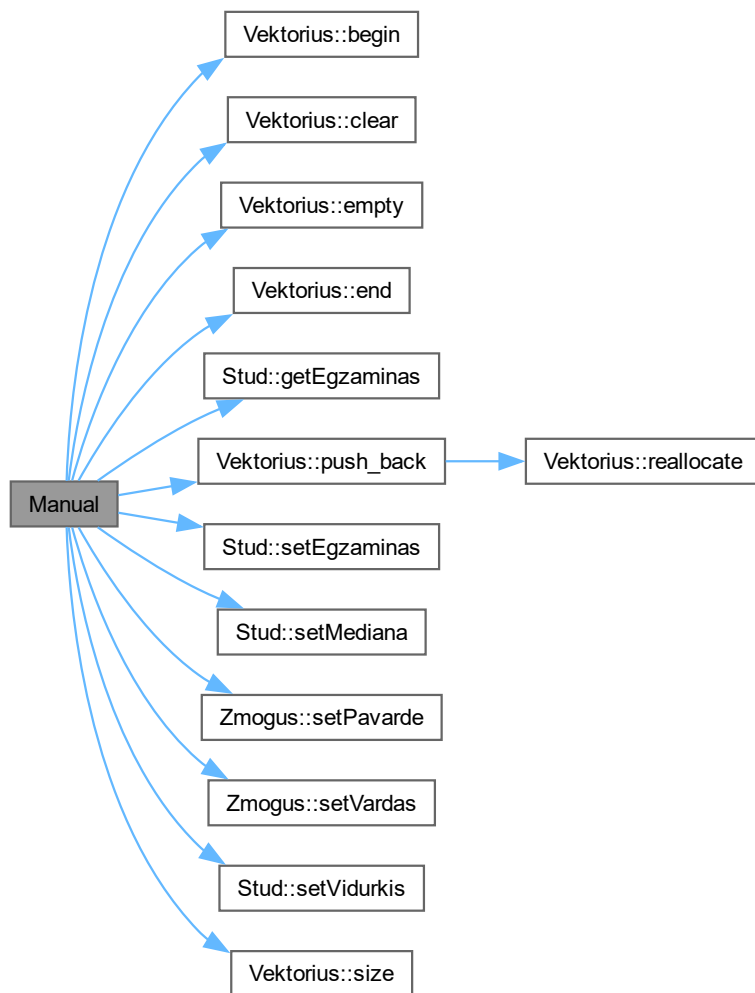
```
void Manual (  
    Stud & laik,  
    Vektorius< Zmogus * > & grupe)
```

Funkcija studentų duomenų įvedimui rankiniu būdu.

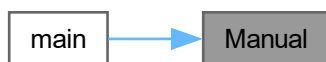
Parametrai

<i>laik</i>	Laikinas studento objektas.
<i>grupe</i>	Studentų grupė.

Funkcijos kvietimo grafas:



Here is the caller graph for this function:



6.6.1.5 Rusiuoti()

```
void Rusiuoti (
```

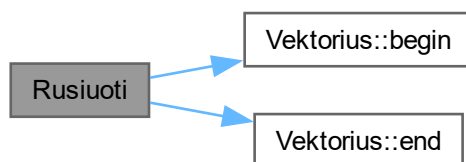
```
Vektorius< Zmogus * > & grupe,
char rusiavimas,
char gal,
double & TotalTime)
```

Funkcija studentų rikiavimui pagal pasirinktą kriterijų.

Parametrai

<i>grupe</i>	Studentų grupė.
<i>rusiavimas</i>	Rikiavimo kriterijus ('v' - vardas, 'p' - pavardė, 'g' - galutinis rezultatas).
<i>gal</i>	Pasirinkimas, ar naudoti vidurkį ('0') ar medianą ('1').
<i>TotalTime</i>	Bendras operacijos laikas.

Funkcijos kvietimo grafas:



Here is the caller graph for this function:



6.6.1.6 Semi()

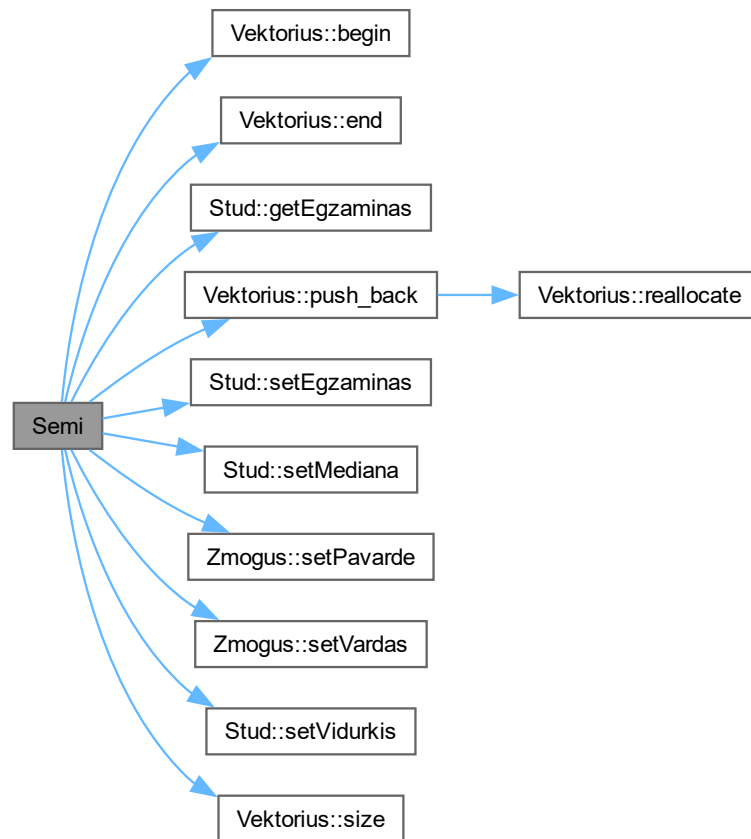
```
void Semi (
    Stud & laik,
    Vektorius< Zmogus * > & grupe)
```

Funkcija studentų duomenų įvedimui pusiau automatinio būdu.

Parametrai

<i>laik</i>	Laikinas studento objektas.
<i>grupe</i>	Studentų grupė.

Funkcijos kvietimo grafas:



Here is the caller graph for this function:



6.6.1.7 Skirstymas1()

```

void Skirstymas1 (
    Vektorius< Zmogus * > & grupe,
    char gal,
    Vektorius< Stud > & vargsiukai,

```



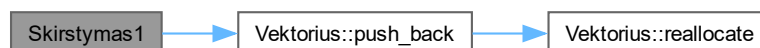
```
Vektorius< Stud > & galvociai,
double & TotalTime)
```

Funkcija studentų skirstymui į dvi grupes pagal vidurkį arba medianą.

Parametrai

<i>grupe</i>	Studentų grupė.
<i>gal</i>	Pasirinkimas, ar naudoti vidurkį ('0') ar medianą ('1').
<i>vargsiukai</i>	Grupė studentų, kurių rezultatai mažesni nei 5.
<i>galvociai</i>	Grupė studentų, kurių rezultatai didesni arba lygūs 5.
<i>TotalTime</i>	Bendras operacijos laikas.

Funkcijos kvietimo grafas:



Here is the caller graph for this function:



6.6.1.8 Skirstymas2()

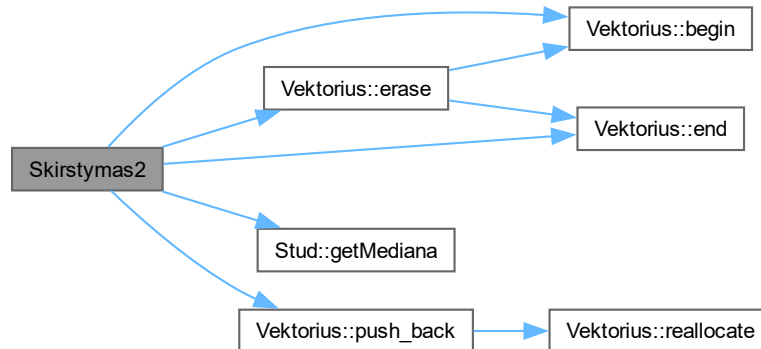
```
void Skirstymas2 (
    Vektorius< Zmogus * > & grupe,
    char gal,
    Vektorius< Stud > & vargsiukai,
    double & TotalTime)
```

Funkcija studentų skirstymui į dvi grupes su pašalinimu iš pradinės grupės.

Parametrai

<i>grupe</i>	Studentų grupė.
<i>gal</i>	Pasirinkimas, ar naudoti vidurkį ('0') ar medianą ('1').
<i>vargsiukai</i>	Grupė studentų, kurių rezultatai mažesni nei 5.
<i>TotalTime</i>	Bendras operacijos laikas.

Funkcijos kvietimo grafas:



Here is the caller graph for this function:



6.6.1.9 Skirstymas3()

```

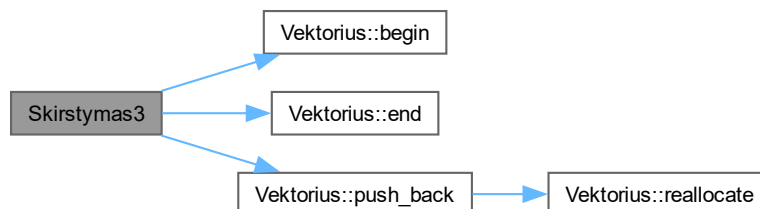
void Skirstymas3 (
    Vektorius< Zmogus * > & grupe,
    char gal,
    Vektorius< Stud > & vargsiukai,
    Vektorius< Stud > & galvociai,
    double & TotalTime)
  
```

Funkcija studentų skirstymui į dvi grupes naudojant `std::partition`.

Parametrai

<i>grupe</i>	Studentų grupė.
<i>gal</i>	Pasirinkimas, ar naudoti vidurkį ('0') ar medianą ('1').
<i>vargsiukai</i>	Grupė studentų, kurių rezultatai mažesni nei 5.
<i>galvociai</i>	Grupė studentų, kurių rezultatai didesni arba lygūs 5.
<i>TotalTime</i>	Bendras operacijos laikas.

Funkcijos kvietimo grafas:



Here is the caller graph for this function:

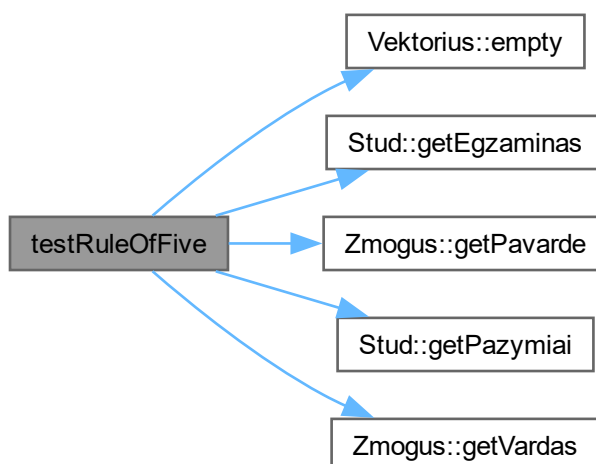


6.6.1.10 testRuleOfFive()

```
void testRuleOfFive ()
```

Funkcija Rule of Five taisyklės testavimui.

Funkcijos kvietimo grafas:



Here is the caller graph for this function:



6.7 header.h

Eiti į šio failo dokumentaciją.

```

00001 #pragma once
00002
00003 #include "dll_header.h"
00004 #include "vector.h"
00005 #include <iostream>
00006 #include <iomanip>
00007 #include <sstream>
00008 #include <conio.h>
00009 #include <string>
00010 #include <ctime>
00011 #include <cstdlib>
00012 #include <fstream>
00013 #include <chrono>
00014 #include <numeric>
00015 #include <cassert>
00016
00017 using std::cin;
00018 using std::cout;
00019 using std::endl;
00020 using std::fixed;
00021 using std::ifstream;
00022 using std::ofstream;
00023 using std::setprecision;
00024 using std::setw;
00025 using std::sort;
00026 using std::string;
00027 using std::to_string;
00028
00032 class Zmogus
00033 {
00034 protected:
00035     string vard, pav;
00036
00037 public:
00041     Zmogus() : vard(""), pav("") {}
00042
00049     Zmogus(const string &vardas, const string &pavarde) : vard(vardas), pav(pavarde) {}
00050
00054     virtual ~Zmogus() = default;
00055
00059     inline string getVardas() const { return vard; }
00060     inline string getPavarde() const { return pav; }
00061
00065     inline void setVardas(const string &vardas) { vard = vardas; }
00066     inline void setPavarde(const string &pavarde) { pav = pavarde; }
00067
00074     virtual void Skaityti(Vektorius<Zmogus *> &grupe, double &TotalTime) = 0;
00075 };
00076
00080 class Stud : public Zmogus
00081 {
00082 private:
00083     Vektorius<int> paz;
00084     int egz;
00085     double vid;
00086     double med;
00087
00088 public:
00092     Stud() : Zmogus(), egz(0), vid(0.0), med(0.0) {}
00093
00102     Stud(const string &vardas, const string &pavarde, const Vektorius<int> &pazymiai, int egzaminas)
00103         : Zmogus(vardas, pavarde), paz(pazymiai), egz(egzaminas), vid(0.0), med(0.0) {}
00104
  
```

```

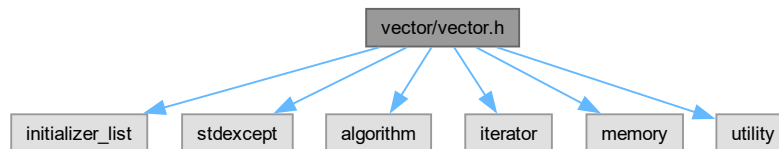
00108     ~Stud();
00109
00115     Stud(const Stud &other)
00116     : Zmogus(other.vard, other.pav), paz(other.paz), egz(other.egz), vid(other.vid),
med(other.med) {}
00117
00124     Stud &operator=(const Stud &other)
00125     {
00126         if (this == &other)
00127             return *this;
00128         Zmogus::operator=(other);
00129         paz = other.paz;
00130         egz = other.egz;
00131         vid = other.vid;
00132         med = other.med;
00133         return *this;
00134     }
00135
00141     Stud(Stud &&other) noexcept
00142     : Zmogus(std::move(other.vard), std::move(other.pav)), paz(std::move(other.paz)),
egz(other.egz), vid(other.vid), med(other.med)
00143     {
00144         other.vard.clear();
00145         other.pav.clear();
00146         other.egz = 0;
00147         other.vid = 0.0;
00148         other.med = 0.0;
00149     }
00150
00151
00158     Stud &operator=(Stud &&other) noexcept
00159     {
00160         if (this == &other)
00161             return *this;
00162         Zmogus::operator=(std::move(other));
00163         paz = std::move(other.paz);
00164         egz = other.egz;
00165         vid = other.vid;
00166         med = other.med;
00167
00168         other.vard.clear();
00169         other.pav.clear();
00170         other.egz = 0;
00171         other.vid = 0.0;
00172         other.med = 0.0;
00173         return *this;
00174     }
00175
00179     inline Vektorius<int> getPazymiai() const { return paz; }
00180     inline int getEgzaminas() const { return egz; }
00181     inline double getVidurkis() const { return vid; }
00182     inline double getMediana() const { return med; }
00183
00187     inline void setPazymiai(const Vektorius<int> &pazymiai) { paz = pazymiai; }
00188     inline void setEgzaminas(int egzaminas) { egz = egzaminas; }
00189     inline void setVidurkis(double vidurkis) { vid = vidurkis; }
00190     inline void setMediana(double mediana) { med = mediana; }
00191
00198     void Skaityti(Vektorius<Zmogus *> &grupe, double &TotalTime) override;
00199 };
00200
00207 void Manual(Stud &laik, Vektorius<Zmogus *> &grupe);
00208
00215 void Semi(Stud &laik, Vektorius<Zmogus *> &grupe);
00216
00222 void Auto(Vektorius<Zmogus *> &grupe);
00223
00231 void Ekrane(Vektorius<Zmogus *> &grupe, char gal, double &TotalTime);
00232
00242 void Skirstymas1(Vektorius<Zmogus *> &grupe, char gal, Vektorius<Stud> &vargsiukai, Vektorius<Stud>
&galvociai, double &TotalTime);
00243
00252 void Skirstymas2(Vektorius<Zmogus *> &grupe, char gal, Vektorius<Stud> &vargsiukai, double
&TotalTime);
00253
00263 void Skirstymas3(Vektorius<Zmogus *> &grupe, char gal, Vektorius<Stud> &vargsiukai, Vektorius<Stud>
&galvociai, double &TotalTime);
00264
00273 void Rusiuoti(Vektorius<Zmogus *> &grupe, char rusiavimas, char gal, double &TotalTime);
00274
00280 void GeneruotiFaila(double &TotalTime);
00281
00285 void testRuleOfFive();

```

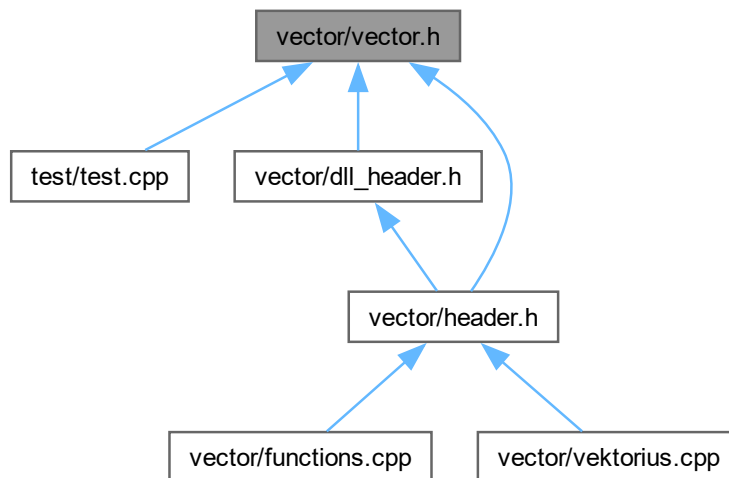
6.8 vector/vector.h Failo Nuoroda

```
#include <initializer_list>
#include <stdexcept>
#include <algorithm>
#include <iterator>
#include <memory>
#include <utility>
```

Įtraukimo priklausomybių diagrama vector.h:



Šis grafas rodo, kuris failas tiesiogiai ar netiesiogiai įtraukia šį failą:



Klasės

- class [Vektorius< T >](#)

Funkcijos

- template<typename T>
bool [operator==](#) (const [Vektorius< T >](#) &lhs, const [Vektorius< T >](#) &rhs)
- template<typename T>
bool [operator!=](#) (const [Vektorius< T >](#) &lhs, const [Vektorius< T >](#) &rhs)
- template<typename T>
bool [operator<](#) (const [Vektorius< T >](#) &lhs, const [Vektorius< T >](#) &rhs)

- `template<typename T>`
`bool operator> (const Vektorius< T > &lhs, const Vektorius< T > &rhs)`
- `template<typename T>`
`bool operator<= (const Vektorius< T > &lhs, const Vektorius< T > &rhs)`
- `template<typename T>`
`bool operator>= (const Vektorius< T > &lhs, const Vektorius< T > &rhs)`

6.8.1 Funkcijos Dokumentacija

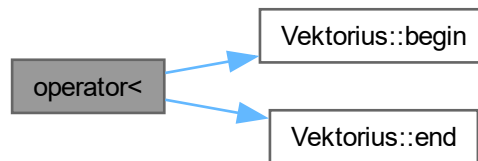
6.8.1.1 operator"!=()"

```
template<typename T>
bool operator!= (
    const Vektorius< T > & lhs,
    const Vektorius< T > & rhs)
```

6.8.1.2 operator<()

```
template<typename T>
bool operator< (
    const Vektorius< T > & lhs,
    const Vektorius< T > & rhs)
```

Funkcijos kvietimo grafas:



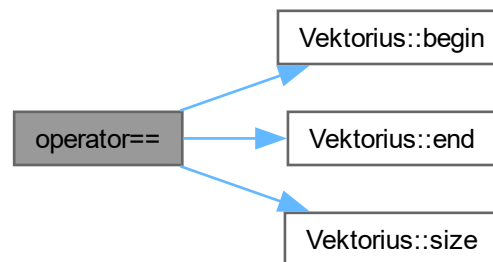
6.8.1.3 operator<=()

```
template<typename T>
bool operator<= (
    const Vektorius< T > & lhs,
    const Vektorius< T > & rhs)
```

6.8.1.4 operator==()

```
template<typename T>
bool operator== (
    const Vektorius< T > & lhs,
    const Vektorius< T > & rhs)
```

Funkcijos kvietimo grafas:



6.8.1.5 operator>()

```

template<typename T>
bool operator> (
    const Vektorius< T > & lhs,
    const Vektorius< T > & rhs)
  
```

6.8.1.6 operator>=()

```

template<typename T>
bool operator>= (
    const Vektorius< T > & lhs,
    const Vektorius< T > & rhs)
  
```

6.9 vector.h

[Eiti į šio failo dokumentaciją.](#)

```

00001 #pragma once
00002 #include <initializer_list>
00003 #include <stdexcept>
00004 #include <algorithm>
00005 #include <iterator>
00006 #include <memory>
00007 #include <utility>
00008
00009 template <typename T>
00010 class Vektorius
00011 {
00012 public:
00013     // Member types
00014     using value_type = T;
00015     using size_type = std::size_t;
00016     using iterator = T *;
00017     using const_iterator = const T *;
00018
00019 private:
00020     std::unique_ptr<T[]> data_; // Automatic memory management using std::unique_ptr
00021     size_type size_;           // Current number of elements
00022     size_type capacity_;       // Maximum number of elements that can be stored
00023
00024     // Function to reallocate data to a new, larger array
00025     void reallocate(size_type new_capacity)
00026     {
00027         std::unique_ptr<T[]> new_data = std::make_unique<T[]>(new_capacity);
00028         std::move(data_.get(), data_.get() + size_, new_data.get());
00029         data_ = std::move(new_data);
00030         capacity_ = new_capacity;
00031     }
  
```



```

00032
00033 public:
00034     // Konstruktoriai
00035     Vektorius() : data_(nullptr), size_(0), capacity_(0) {} // Default konstruktorius
00036     explicit Vektorius(size_type count, const T &value = T()) : size_(count), capacity_(count)
00037     {
00038         data_ = std::make_unique<T[]>(capacity_);
00039         std::fill(data_.get(), data_.get() + size_, value);
00040     }
00041     Vektorius(std::initializer_list<T> init) : Vektorius(init.size())
00042     {
00043         std::copy(init.begin(), init.end(), data_.get());
00044     }
00045     Vektorius(const Vektorius &other) : size_(other.size_), capacity_(other.capacity_)
00046     {
00047         data_ = std::make_unique<T[]>(capacity_);
00048         std::copy(other.data_.get(), other.data_.get() + size_, data_.get());
00049     }
00050     Vektorius(Vektorius &&other) noexcept : data_(std::move(other.data_)), size_(other.size_),
00051     capacity_(other.capacity_)
00052     {
00053         other.size_ = 0;
00054         other.capacity_ = 0;
00055     }
00056     // Assignment operatoriai
00057     Vektorius &operator=(const Vektorius &other)
00058     {
00059         if (this != &other)
00060         {
00061             data_ = std::make_unique<T[]>(other.capacity_);
00062             size_ = other.size_;
00063             capacity_ = other.capacity_;
00064             std::copy(other.data_.get(), other.data_.get() + size_, data_.get());
00065         }
00066         return *this;
00067     }
00068     Vektorius &operator=(Vektorius &&other) noexcept
00069     {
00070         if (this != &other)
00071         {
00072             data_ = std::move(other.data_);
00073             size_ = other.size_;
00074             capacity_ = other.capacity_;
00075             other.size_ = 0;
00076             other.capacity_ = 0;
00077         }
00078         return *this;
00079     }
00080
00081     // Member funkcijos
00082     size_type size() const noexcept { return size_; } // Returns the current number of
00083     elements
00084     size_type capacity() const noexcept { return capacity_; } // Returns the capacity
00085     bool empty() const noexcept { return size_ == 0; } // Checks if the vector is empty
00086
00087     // Adds a new element to the end of the vector
00088     void push_back(const T &value)
00089     {
00090         if (size_ == capacity_)
00091         {
00092             reallocate(capacity_ == 0 ? 1 : capacity_ * 2);
00093         }
00094         data_[size_++] = value;
00095     }
00096     // Removes the last element
00097     void pop_back()
00098     {
00099         if (size_ > 0)
00100         {
00101             --size_;
00102         }
00103     }
00104
00105     // Clears all elements
00106     void clear() noexcept
00107     {
00108         size_ = 0;
00109     }
00110     void swap(Vektorius &other) noexcept
00111     {
00112         std::swap(data_, other.data_);
00113         std::swap(size_, other.size_);
00114         std::swap(capacity_, other.capacity_);
00115     }
00116

```

```

00117 // Changes the size of the vector
00118 void resize(size_type new_size, const T &value = T())
00119 {
00120     if (new_size > capacity_)
00121     {
00122         reallocate(new_size);
00123     }
00124     if (new_size > size_)
00125     {
00126         std::fill(data_.get() + size_, data_.get() + new_size, value);
00127     }
00128     size_ = new_size;
00129 }
00130
00131 // Reserves memory for the specified number of elements
00132 void reserve(size_type new_capacity)
00133 {
00134     if (new_capacity > capacity_)
00135     {
00136         reallocate(new_capacity);
00137     }
00138 }
00139
00140 // Removes the element at the specified position and returns an iterator to the next element
00141 iterator erase(iterator pos)
00142 {
00143     if (pos < begin() || pos >= end())
00144     {
00145         throw std::out_of_range("Iterator out of range");
00146     }
00147     std::move(pos + 1, end(), pos); // Shift elements to the left
00148     --size_;                       // Decrease the size
00149     return pos;
00150 }
00151
00152 // Element access operators
00153 T &operator[](size_type index)
00154 {
00155     if (index >= size_)
00156         throw std::out_of_range("Index out of bounds");
00157     return data_[index];
00158 }
00159
00160 const T &operator[](size_type index) const
00161 {
00162     if (index >= size_)
00163         throw std::out_of_range("Index out of bounds");
00164     return data_[index];
00165 }
00166
00167 T &at(size_type index)
00168 {
00169     if (index >= size_)
00170         throw std::out_of_range("Index out of bounds");
00171     return data_[index];
00172 }
00173
00174 const T &at(size_type index) const
00175 {
00176     if (index >= size_)
00177         throw std::out_of_range("Index out of bounds");
00178     return data_[index];
00179 }
00180
00181 // Iterators
00182 iterator begin() noexcept { return data_.get(); }
00183 const_iterator begin() const noexcept { return data_.get(); }
00184 iterator end() noexcept { return data_.get() + size_; }
00185 const_iterator end() const noexcept { return data_.get() + size_; }
00186 };
00187
00188 // Non-member funkcijos
00189 template <typename T>
00190 bool operator==(const Vektorius<T> &lhs, const Vektorius<T> &rhs)
00191 {
00192     return lhs.size() == rhs.size() && std::equal(lhs.begin(), lhs.end(), rhs.begin());
00193 }
00194
00195 template <typename T>
00196 bool operator!=(const Vektorius<T> &lhs, const Vektorius<T> &rhs)
00197 {
00198     return !(lhs == rhs);
00199 }
00200
00201 template <typename T>
00202 bool operator<(const Vektorius<T> &lhs, const Vektorius<T> &rhs)
00203 {

```

```

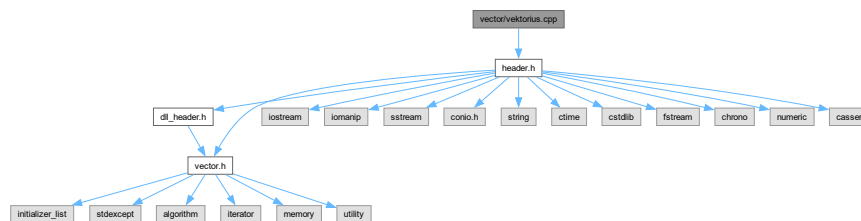
00204     return std::lexicographical_compare(lhs.begin(), lhs.end(), rhs.begin(), rhs.end());
00205 }
00206
00207 template <typename T>
00208 bool operator<(const Vektorius<T> &lhs, const Vektorius<T> &rhs)
00209 {
00210     return rhs < lhs;
00211 }
00212
00213 template <typename T>
00214 bool operator<=(const Vektorius<T> &lhs, const Vektorius<T> &rhs)
00215 {
00216     return !(rhs < lhs);
00217 }
00218
00219 template <typename T>
00220 bool operator>(const Vektorius<T> &lhs, const Vektorius<T> &rhs)
00221 {
00222     return !(lhs < rhs);
00223 }

```

6.10 vector/vektorius.cpp Failo Nuoroda

```
#include "header.h"
```

Įtraukimo priklausomybių diagrama vektorius.cpp:



Funkcijos

- int **main** ()

Pagrindinė programos funkcija.

6.10.1 Funkcijos Dokumentacija

6.10.1.1 main()

```
int main ()
```

Pagrindinė programos funkcija.

Ši funkcija leidžia vartotojui pasirinkti įvairias operacijas, tokias kaip studentų duomenų generavimas, įvedimas, rikiavimas, skirstymas ir išvedimas į ekraną arba failą.

Gražina

int Programos vykdymo rezultatas.

Tikrinama, ar vartotojas nori testuoti Rule of Five taisyklę.

Tikrinama, ar vartotojas nori sugeneruoti studentų failą.

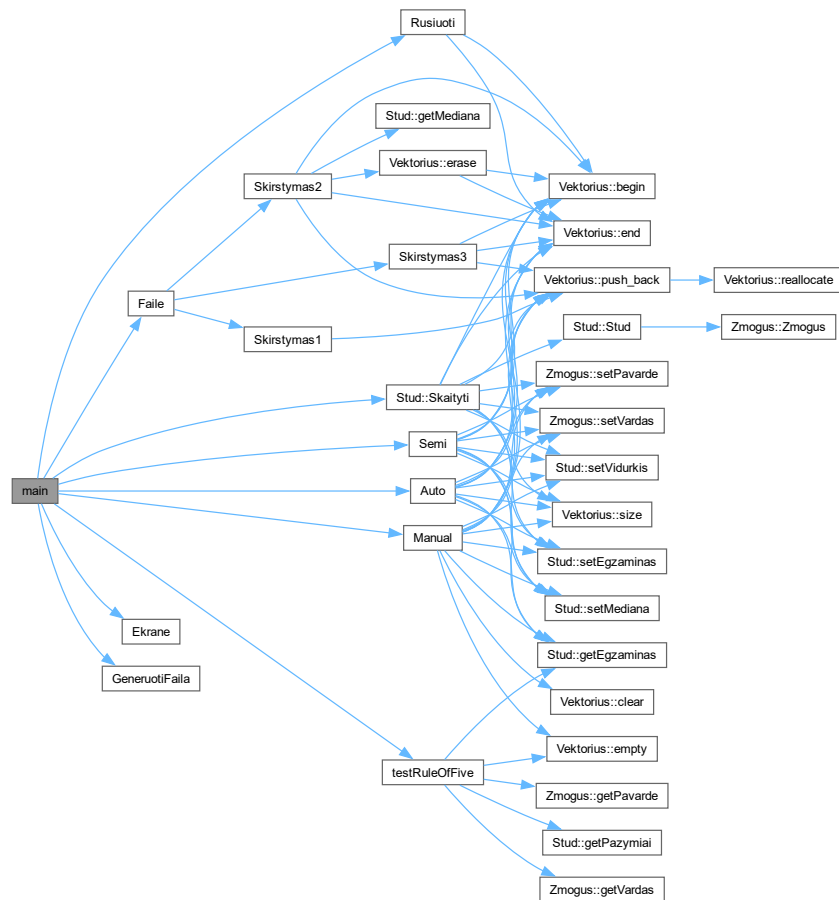
Leidžiama pasirinkti studentų duomenų įvedimo būdą.

Pasirenkama, ar naudoti vidurkį, ar medianą.

Pasirenkamas rikiavimo kriterijus.

Pasirenkama, ar duomenis išvesti į ekraną, ar į failą.

Išvedamas bendras programos veikimo laikas. Funkcijos kvietimo grafas:



Rodyklė

- ~Stud
 - Stud, [14](#)
- ~Zmogus
 - Zmogus, [35](#)
- at
 - Vektorius< T >, [23](#)
- Auto
 - functions.cpp, [49](#)
 - header.h, [59](#)
- begin
 - Vektorius< T >, [23](#)
- capacity
 - Vektorius< T >, [24](#)
- capacity_
 - Vektorius< T >, [33](#)
- CATCH_CONFIG_MAIN
 - test.cpp, [40](#)
- clear
 - Vektorius< T >, [24](#)
- const_iterator
 - Vektorius< T >, [21](#)
- data_
 - Vektorius< T >, [33](#)
- DLL_API
 - dll_header.h, [46](#)
- dll_header.h
 - DLL_API, [46](#)
 - Faile, [47](#)
- egz
 - Stud, [19](#)
- Ekrane
 - functions.cpp, [50](#)
 - header.h, [60](#)
- empty
 - Vektorius< T >, [25](#)
- end
 - Vektorius< T >, [25](#)
- erase
 - Vektorius< T >, [26](#)
- EXPORTING_DLL
 - functions.cpp, [48](#)
- Faile
 - dll_header.h, [47](#)
 - functions.cpp, [50](#)

- functions.cpp
 - Auto, [49](#)
 - Ekrane, [50](#)
 - EXPORTING_DLL, [48](#)
 - Faile, [50](#)
 - GeneruotiFaila, [51](#)
 - Manual, [51](#)
 - Rusiuoti, [52](#)
 - Semi, [53](#)
 - Skirstymas1, [54](#)
 - Skirstymas2, [55](#)
 - Skirstymas3, [56](#)
 - testRuleOfFive, [57](#)
- GeneruotiFaila
 - functions.cpp, [51](#)
 - header.h, [61](#)
- getEgzaminas
 - Stud, [15](#)
- getMediana
 - Stud, [15](#)
- getPavarde
 - Zmogus, [35](#)
- getPazymiai
 - Stud, [15](#)
- getVardas
 - Zmogus, [35](#)
- getVidurkis
 - Stud, [16](#)
- header.h
 - Auto, [59](#)
 - Ekrane, [60](#)
 - GeneruotiFaila, [61](#)
 - Manual, [61](#)
 - Rusiuoti, [62](#)
 - Semi, [63](#)
 - Skirstymas1, [64](#)
 - Skirstymas2, [65](#)
 - Skirstymas3, [66](#)
 - testRuleOfFive, [67](#)
- iterator
 - Vektorius< T >, [21](#)
- main
 - vektorius.cpp, [75](#)
- Manual
 - functions.cpp, [51](#)
 - header.h, [61](#)

- med
 - Stud, 19
- operator!=
 - vector.h, 71
- operator<
 - vector.h, 71
- operator<=
 - vector.h, 71
- operator>
 - vector.h, 72
- operator>=
 - vector.h, 72
- operator=
 - Stud, 16
 - Vektorius< T >, 27
- operator==
 - vector.h, 71
- operator[]
 - Vektorius< T >, 27
- pav
 - Zmogus, 37
- paz
 - Stud, 20
- pop_back
 - Vektorius< T >, 28
- Programos versijos, 1
- push_back
 - Vektorius< T >, 28
- README.md, 39
- reallocate
 - Vektorius< T >, 29
- reserve
 - Vektorius< T >, 30
- resize
 - Vektorius< T >, 30
- Rusiuti
 - functions.cpp, 52
 - header.h, 62
- Semi
 - functions.cpp, 53
 - header.h, 63
- setEgzaminas
 - Stud, 17
- setMediana
 - Stud, 17
- setPavarde
 - Zmogus, 36
- setPazymiai
 - Stud, 18
- setVardas
 - Zmogus, 36
- setVidurkis
 - Stud, 18
- size
 - Vektorius< T >, 31
- size_
 - Vektorius< T >, 33
- size_type
 - Vektorius< T >, 21
- Skaityti
 - Stud, 18
 - Zmogus, 37
- Skirstymas1
 - functions.cpp, 54
 - header.h, 64
- Skirstymas2
 - functions.cpp, 55
 - header.h, 65
- Skirstymas3
 - functions.cpp, 56
 - header.h, 66
- Stud, 11
 - ~Stud, 14
 - egz, 19
 - getEgzaminas, 15
 - getMediana, 15
 - getPazymiai, 15
 - getVidurkis, 16
 - med, 19
 - operator=, 16
 - paz, 20
 - setEgzaminas, 17
 - setMediana, 17
 - setPazymiai, 18
 - setVidurkis, 18
 - Skaityti, 18
 - Stud, 13, 14
 - vid, 20
- swap
 - Vektorius< T >, 32
- test.cpp
 - CATCH_CONFIG_MAIN, 40
 - TEST_CASE, 40–45
- test/test.cpp, 39
- TEST_CASE
 - test.cpp, 40–45
- testRuleOfFive
 - functions.cpp, 57
 - header.h, 67
- value_type
 - Vektorius< T >, 21
- vard
 - Zmogus, 37
- vector.h
 - operator!=, 71
 - operator<, 71
 - operator<=, 71
 - operator>, 72
 - operator>=, 72
 - operator==, 71
- vector/dll_header.h, 46, 47
- vector/functions.cpp, 48

- vector/header.h, [58](#), [68](#)
- vector/vector.h, [70](#), [72](#)
- vector/vektorius.cpp, [75](#)
- Vektorius
 - Vektorius< T >, [21](#), [22](#)
- Vektorius< T >, [20](#)
 - at, [23](#)
 - begin, [23](#)
 - capacity, [24](#)
 - capacity_, [33](#)
 - clear, [24](#)
 - const_iterator, [21](#)
 - data_, [33](#)
 - empty, [25](#)
 - end, [25](#)
 - erase, [26](#)
 - iterator, [21](#)
 - operator=, [27](#)
 - operator[], [27](#)
 - pop_back, [28](#)
 - push_back, [28](#)
 - reallocate, [29](#)
 - reserve, [30](#)
 - resize, [30](#)
 - size, [31](#)
 - size_, [33](#)
 - size_type, [21](#)
 - swap, [32](#)
 - value_type, [21](#)
 - Vektorius, [21](#), [22](#)
- vektorius.cpp
 - main, [75](#)
- vid
 - Stud, [20](#)
- Zmogus, [33](#)
 - ~Zmogus, [35](#)
 - getPavarde, [35](#)
 - getVardas, [35](#)
 - pav, [37](#)
 - setPavarde, [36](#)
 - setVardas, [36](#)
 - Skaityti, [37](#)
 - vard, [37](#)
 - Zmogus, [34](#), [35](#)